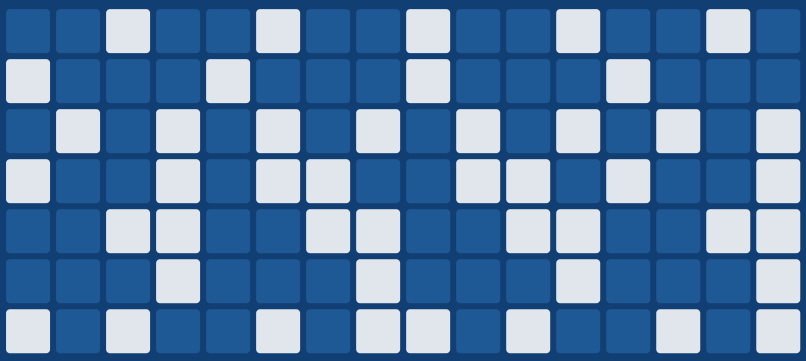


Алгоритмын долоон бодлого

Бодолт, баталгаа, кодын тайлбар

OnlineContest хавтасны A–G бодлого бүрийг ажиглалтаас эхлээд ажилладаг C++ код хүртэл алхам алхмаар тайлбарласан гарын авлага.

2026 оны 11-р сар

Агуулга

Өмнөх үг 4

Бүлэг 1 A. Numbers on the Board 6

1.1 Бодлого 6
1.2 Нөхцөлийг хувиргах 6
1.3 Тохиолдлуудыг шинжлэх 7
1.4 Код 8
1.5 Хугацаа ба анхаарах зүйл 8

Бүлэг 2 B. Rhythmic String 9

2.1 Бодлого 9
2.2 Жишээгээр ойлгох 9
2.3 Гол теорем 10
2.4 Алгоритм ба жишээ 10
2.5 Код 11
2.6 Хугацаа ба анхаарах зүйл 11

Бүлэг 3 C. Big XOR 13

3.1 Бодлого 13
3.2 Зорилтын утгыг тодорхойлох 13
3.3 Давхарлах үйлдлийг XOR-оор илэрхийлэх 13
3.4 Шугаман суурь 14
3.5 Алгоритм ба код 15
3.6 Яагаад хариу заавал олддог вэ 17
3.7 Илүү энгийн хувилбар 17
3.8 Хугацаа ба анхаарах зүйл 18

Бүлэг 4 D. Black, Gray and White 20

4.1 Бодлого 20
4.2 Эцсийн өнгө 20
4.3 Хар оройн тоо ба нэр дэвшигч k 20
4.4 Саарал орой 21
4.5 Алгоритм 22
4.6 Жишээгээр мөрдөх 23
4.7 Код 24
4.8 Хэшгүй хувилбар 26
4.9 Хугацаа ба анхаарах зүйл 26

Бүлэг 5 E. Partition of a Binary String 28

5.1 Бодлого 28
5.2 Жишээгээр ойлгох 28
5.3 Холимог олонлог ямар байх вэ 28
5.4 Өртгийг хялбарчлах 29
5.5 Ээлжилсэн хэсгүүд 29
5.6 Оновчтой хуваалтыг тоолох 30
5.7 Беллийн тоог тооцох 31
5.8 Алгоритм ба жишээ 31
5.9 Код 32
5.10 Хугацаа ба анхаарах зүйл 33

Бүлэг 6 F. Minimize the Graph 35

6.1 Бодлого 35
6.2 Хамгийн цөөн ирмэгтэй граф 35

6.3 Аль ирмэгийг үлдээх вэ 36
6.4 Алгоритм ба хэрэгслүүд 36
6.5 Код 38
6.6 Хугацаа ба анхаарах зүйл 39
6.7 Нэгэн алдааны түүх 39

Бүлэг 7 G. Order Statistics on the Graph 41
7.1 Бодлого 41
7.2 Жишээгээр ойлгох 41
7.3 Босгоор задлах 42
7.4 Утгыг битүүдээс сэргээх 42
7.5 Хоёрын зэргээр үсрэх 43
7.6 Алгоритм ба жишээ 43
7.7 Код 44
7.8 Хугацаа ба анхаарах зүйл 45

Хавсралт А. Шалгалтын тайлан 47

Хавсралт Б. Нэр томьёоны толь 49

Хавсралт В. Дасгалын хариу 50

Өмнөх үг

Энэ гарын авлага нь `OnlineContest` хавтаст байгаа долоон бодлогын бодолтыг номын хэлбэрээр тайлбарлана. Зорилго нь зөвхөн ажилладаг код үзүүлэх биш. Бодлогыг уншаад ямар ажиглалт хийх, тэр ажиглалт яагаад зөв болохыг хэрхэн батлах, дараа нь түүнийгээ яаж код болгохыг дарааллаар нь харуулна.

Хэнд зориулав. C++ хэлний үндэс (массив, `vector`, давталт), битийн үйлдэл, граф ба модны анхан шатны ойлголттой уншигчид зориулав. Шугаман суурь, Беллийн тоо, Фенвикийн мод зэрэг тусгай хэрэгслийг ашиглагдах бүлэгт нь тайлбарласан.

Бүлэг бүрийн бүтэц

- **Бодлого.** Нөхцөлийг товч, тодорхой тэмдэглэгээтэйгээр өөрийн үгээр дахин бичсэн. Анхны англи текст тухайн `.cpp` файлын сүүлийн коммент дотор байгаа.
- **Гол санаа.** Ажиглалт, лемм, теорем болон тэдгээрийн баталгаа. Цэнхэр хүрээтэй хэсгийг уншихад л шийдлийн хүрээ харагдана.
- **Алгоритм ба жишээ.** Алхмуудыг жагсааж, жижиг жишээн дээр гараар мөрдөнө.
- **Код.** Файл дахь кодыг бүхэлд нь үзүүлж, хэсэг бүрийг тайлбарлана. Мөрийн дугаар нь файлын жинхэнэ мөртэй таарна. Хэсэгчилсэн кодын эхний доголыг уншихад хялбар болгож багасгасан.
- **Хугацаа, анхаарах зүйл, дүгнэлт.** Хурд, түгээмэл алдаа, цаашид бусад бодлогод хэрэглэж болох санаа.

Өнгөний утга. Цэнхэр хүрээ нь тодорхойлолт, лемм, теорем; ногоон нь шинэ хэрэгслийн тайлбар; улбар шар нь анхаарах зүйл; саарал нь дүгнэлт. Бүлэг бүрийн төгсгөлд шалгах дасгал байгаа ба хариу нь Хавсралт В-д бий.

Хэрхэн унших вэ

Бодлого бүрийг уншаад шууд тайлбар руу орохгүй, 15–20 минут өөрөө бодож үзээрэй. Гацвал эхний ажиглалтыг уншаад дахин оролдоорой. Баталгааг алгасаж болох ч эхлээд ойлгож авахыг зөвлөж байна: олон бодлогод алдаа гарах шалтгаан нь “зөв юм шиг” боловч батлагдаагүй таамаг байдаг. Үүний бодит жишээг 6-р бүлгийн төгсгөлд харна.

Кодыг ажиллуулах

```
g++ -O2 -std=c++17 -o A A.cpp
./A < input.txt
```

Бодлогуудын тойм

Бүлэг	Бодлого	Гол санаа	Хугацаа
1	A. Numbers on the Board	Бусдын нийлбэр 0 байх олонлог	$O(1)$
2	B. Rhythmic String	Хамгийн бага элемент бусдаа хуваана	$O(n^2)$
3	C. Big XOR	XOR-ын шугаман систем, тэгш сондгойн чанар	$O(n \log n)$
4	D. Black, Gray and White	k-ийн цөөн нэр дэвшигч, дэд модны хэш	$O(n)$
5	E. Partition of a Binary String	Ээлжилсэн хэсэг, Беллийн тоо	$O(n)$, хүснэгт $O(N^2)$
6	F. Minimize the Graph	Транзитив багасгалт, хамгийн бага гарах хөрш	$O(n \log n)$
7	G. Order Statistics on the Graph	Босгоор битэд задлах, хоёрын зэргээр үсрэх	$O(2^n(n + \log t) + q\ n \log t)$

Тэмдэглэгээ

Тэмдэг	Утга
$a \oplus b$	XOR үйлдэл (C++: `a ^ b`)
$a \mid b$	OR үйлдэл (C++: `a | b`)
$[l, r]$	l-ээс r хүртэлх бүхэл тоонууд, хоёр захаа оруулаад
$\lvert S \rvert$	S олонлогийн элементийн тоо
$a \mid b$	a нь b-г үлдэгдэлгүй хуваана
$u \rightsquigarrow v$	графт u-ээс v рүү зам байна
popcount(x)	x-ийн хоёртын бичлэг дэх 1 битийн тоо
$O(f(n))$	$f(n)$-тэй пропорциональ буюу түүнээс удаашрахгүй өсөх хугацаа
■	баталгааны төгсгөл

БҮЛЭГ 1

A. Numbers on the Board

Самбар дээрх тоонууд

Сэдэв математик ажиглалт, тэгшитгэл хувиргах **Файл** `A.cpp`

Эхний бодлого нь ямар ч өгөгдлийн бүтэц шаардахгүй. Гол зүйл нь нөхцөлийг нэг алхам хувиргаж, хариуг энгийн томьёо хүртэл хялбарчлах явдал юм.

1.1 Бодлого

Самбар дээр l-ээс r хүртэлх бүх бүхэл тоо тус бүр нэг удаа бичигдсэн байна. Самбар дээрх нэг тоо самбарт үлдсэн бүх тооны нийлбэртэй (өөрийгөө оруулаад) тэнцүү бол түүнийг **сайхан тоо** гэж нэрлэе. Хүссэн тоогоо арилгаж болох боловч дор хаяж нэг тоо үлдэх ёстой. Самбар дээр сайхан тоо гаргахын тулд хамгийн цөөндөө хэдэн тоо арилгах вэ?

Оролт: t тест ($t \le 10^4$), тест бүрт l ба r ($-100 \le l \le r \le 100$). **Гаралт:** тест бүрт арилгах тооны хамгийн бага утга. Эх сурвалж: `codeforces.com/contest/2283/problem/A`.

Хүснэгт 1.1. Жишээ тестүүд, үлдээх нэг боломжит сонголтын хамт.

l	r	Хариу	Үлдэх тоонууд	Сайхан тоо
0	0	0	0	0
2	6	4	6	6
−5	−3	2	−5	−5
−2	2	0	−2, −1, 0, 1, 2	0
−2	3	0	−2, −1, 0, 1, 2, 3	3
−2	5	2	−2, −1, 0, 1, 2, 3	3
−6	2	3	−4, −2, −1, 0, 1, 2	−4
0	4	3	0, 4	4

1.2 Нөхцөлийг хувиргах

Үлдсэн тоонуудын олонлогийг K гэе. K дотор сайхан тоо x байна гэдэг нь K-ийн бүх тооны нийлбэр x-тэй тэнцүү гэсэн үг. Энэ тэгшитгэлийн хоёр талаас x-ийг хасвал нэг чухал зүйл харагдана: x-ээс **бусад** үлдсэн тоонуудын нийлбэр 0 байх ёстой.

> **Ажиглалт 1.1.** K дотор сайхан тоо байх нь K-г $\{x\} \cup Z$ хэлбэрээр бичиж болохтой ижил. Энд $x \notin Z$, Z-ийн нийлбэр 0 бөгөөд Z хоосон байж болно.

Ингэснээр бодлого илүү ойлгомжтой болно: **нийлбэр нь 0 байх аль болох том Z олонлог сонгоод, түүн дээр нэг тоо нэмнэ.** Нийт $n = r - l + 1$ тоо байгаа тул хариу нь $n - |K|$.

1.3 Тохиолдлуудыг шинжлэх

Бүх тоо нэг тэмдэгтэй үед

$l > 0$ эсвэл $r < 0$ бол бүх тоо ижил тэмдэгтэй, дунд нь 0 байхгүй. Хоосон биш Z-ийн нийлбэр 0 байж чадахгүй тул $Z = \emptyset$, өөрөөр хэлбэл зөвхөн нэг тоо үлдэнэ. Хариу нь $n - 1 = r - l$.

Самбар дээр 0 байгаа үед

Одоо $l \le 0 \le r$ гэе. $a = -l$, $b = r$, $m = \min(a, b)$ гэж тэмдэглэвэл самбар дээр $-a$-аас b хүртэлх тоонууд байна. Нийлбэр нь 0 байх хамгийн энгийн олонлог бол тэгш хэмтэй $Z = \{-m, \ldots, m\}$ юм. Энэ нь $2m + 1$ тоотой. Үүнээс том Z олдохгүй гэдгийг баталъя.

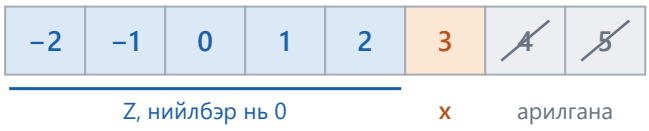
Лемм 1.2. Нийлбэр нь 0 байх $Z \subseteq [-a, b]$ олонлогт m-ээс олон эерэг тоо, m-ээс олон сөрөг тоо байхгүй. Иймд $|Z| \le 2m + 1$.

Баталгаа. Z-д p эерэг, q сөрөг тоо байна гэе. Эерэг тоонууд хоорондоо ялгаатай тул нийлбэр нь дор хаяж $1 + 2 + \ldots + p$ байна. Сөрөг тоонуудын абсолют утгууд нь $1, \ldots, a$ доторх ялгаатай тоонууд тул нийлбэр нь хамгийн ихдээ $1 + 2 + \ldots + a$. Z-ийн нийлбэр 0 учир эдгээр хоёр нийлбэр тэнцүү, иймд $1 + \ldots + p \le 1 + \ldots + a$, эндээс $p \le a$. Мөн $p \le b$ нь шууд гарна. Тэгэхээр $p \le m$. Сөрөг талын хувьд $q \le m$ яг ижилхэн батлагдана. 0 тоог нэмээд $|Z| \le 2m + 1$. ■

Теорем 1.3. $l \le 0 \le r$ үед хариу нь $\max(|l + r| - 1, 0)$.

Баталгаа. $a = b$ бол бүх тоог үлдээе. Нийлбэр нь 0, самбар дээр 0 байгаа тул 0 нь сайхан тоо. Хариу 0.

$a \ne b$ бол Ажиглалт 1.1 ба Лемм 1.2-оос $|K| = |Z| + 1 \le 2m + 2$. Энэ хязгаарт хүрч болно: $Z = \{-m, \ldots, m\}$ гээд x-ээр үлдсэн тоонуудаас дурын нэгийг сонгоно. $a \ne b$ учир ийм тоо заавал бий. $n = a + b + 1$ ба $a + b = 2m + |a - b|$ тул хариу нь $n - (2m + 2) = |a - b| - 1$. Сүүлд нь $a - b = -(l + r)$ гэдгийг орлуулбал $|l + r| - 1$ гарна. Энэ утга $a \ne b$ үед 0-ээс багагүй, $a = b$ үед -1 болох тул $\max(\ldots, 0)$ хоёр тохиолдлыг нэгтгэнэ. ■



Зураг 1.1. $l = -2$, $r = 5$ үед: $m = 2$, $Z = \{-2, \ldots, 2\}$, $x = 3$. Арилгах тоо нь $|l + r| - 1 = 3 - 1 = 2$.

Хүснэгт 1.2. Томьёог жишээн дээр шалгах (Хүснэгт 1.1).

l	r	a	b	m	Z	x	Томьёо	Хариу
−2	2	2	2	2	−2..2 (бүгд)	0	max(0 − 1, 0)	0
−2	3	2	3	2	−2..2	3	|1| − 1	0
−2	5	2	5	2	−2..2	3	|3| − 1	2
−6	2	6	2	2	−2..2	−4	|−4| − 1	3
0	4	0	4	0	{0}	4	|4| − 1	3

1.4 Код

Код 1.1. A.cpp, 1–22-р мөр: бүтэн шийдэл

```cpp
#include<bits/stdc++.h>
using namespace std;
int main(){
    int test;
    cin >> test;

    while(test--){
        int l,r;    cin >> l >> r;
        if(l > 0 && r > 0){
            cout << r - l << '\n';
            continue;
        }

        if(l < 0 && r < 0){
            cout << abs(l) - abs(r) << '\n';
            continue;
        }

        cout << max(abs(r + l ) - 1, 0) << '\n';
    }
    return 0;
}
```

- **9–12-р мөр.** Бүх тоо эерэг бол зөвхөн нэг тоо үлдэнэ: хариу $r - l$.
- **14–17-р мөр.** Бүх тоо сөрөг бол мөн адил. Энд `abs(l) - abs(r)` нь $(-l) - (-r) = r - l$-тэй ижил.
- **19-р мөр.** Үлдсэн тохиолдолд $l \le 0 \le r$ тул Теорем 1.3-ын томьёог шууд хэрэглэнэ. $l + r = 0$ үед `abs(r + l) - 1` нь -1 болдог тул `max(..., 0)` хэрэгтэй.

1.5 Хугацаа ба анхаарах зүйл

Тест бүрт $O(1)$, нийт $O(t)$ хугацаа, нэмэлт санах ой $O(1)$.

> **Анхаарах**
>
> Томьёо $\max(|l + r| - 1, 0)$ нь зөвхөн $l \le 0 \le r$ үед зөв. Жишээ нь $(l, r) = (2, 6)$ дээр хэрэглэвэл 7 гарах боловч жинхэнэ хариу 4. Тиймээс кодод эхний хоёр нөхцөлийг заавал шалгана.

> **Дүгнэлт**
>
> “Нэг тоо нь нийт нийлбэртэй тэнцүү” гэсэн нөхцөлийг “бусдын нийлбэр 0” гэж хувиргасан нь гол алхам байв. Тэгшитгэлийг хялбарчилж үзэх нь энгийн боловч үр дүнтэй арга.

> **Дасгал**
>
> **1.1.** $(l, r) = (-3, 7)$ үед хариу хэд вэ? Үлдэх тоонуудын нэг жишээг бич.
>
> **1.2.** $(l, r) = (-1, 1)$ үед аль тоо сайхан болох вэ? Хэдэн тоо арилгах вэ?

БҮЛЭГ 2

B. Rhythmic String

Ритмтэй мөр

Сэдэв тооны онол, тэмдэгт мөрийн үе **Файл** `B.cpp`

Мөрийн урт 10^{18} тул мөрийг шууд үүсгэх боломжгүй. Гэхдээ ритмтэй мөр маш энгийн бүтэцтэй байдаг: үүнийг баталснаар бодлого хуваагдах чанарыг тоолох асуудал болно.

2.1 Бодлого

Урт нь m-тэй хоёртын мөр s-ийн хувьд $1 \le k < m$ бөгөөд бүх боломжит i-д $s_i = s_{i+k}$ байвал k-г мөрийн **үе** гэнэ. Хамгийн бага үеийг p гэж тэмдэглэе. Мөрийн эхний p тэмдэгтэд яг нэг 1 байвал мөрийг **ритмтэй** гэнэ.

n элементтэй a массив өгөгдөнө. 10^{18} урттай бүгд 0 мөрөөс эхлээд, a-ийн элемент бүрийн хувьд түүнд хуваагдах бүх байрлалыг 1 болгоно. Мөрийг үүсгэхийн өмнө a-аас дурын элементүүдийг хасаж болно. Мөр ритмтэй болохын тулд хамгийн цөөндөө хэдэн элемент хасах вэ?

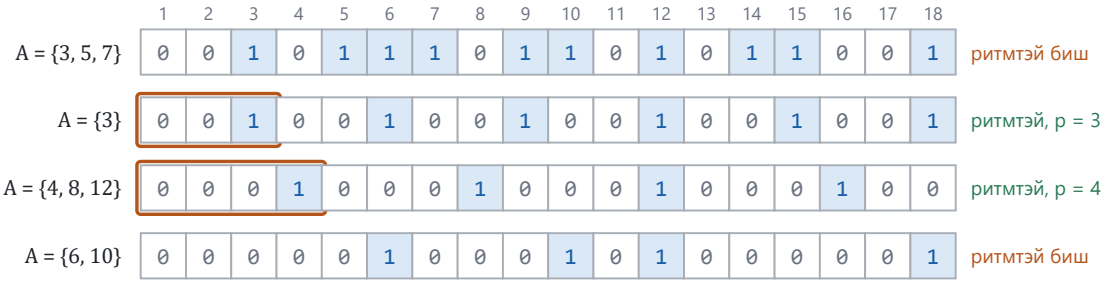
Хязгаар: $n \le 5000$, бүх тестийн n-ийн нийлбэр 5000-аас хэтрэхгүй, $1 \le a_i \le 10^9$.

Хүснэгт 2.1. Жишээ тестүүд.

a	Хариу	Үлдээх элементүүд	Үе
[3, 7, 5, 3]	2	3, 3	3
[4, 8, 12]	0	4, 8, 12	4
[6, 10, 20, 30]	1	10, 20, 30	10
[2, 3, 5]	2	дурын нэг элемент	тэр элемент

2.2 Жишээгээр ойлгох

Зураг 2.1-д хэд хэдэн A олонлогоос үүсэх мөрийн эхний хэсгийг үзүүлэв. Хоёр ажиглалт шууд харагдана. Нэгдүгээрт, $A = \{4, 8, 12\}$ үед 8 ба 12 нь мөрийг огт өөрчлөхгүй, учир нь тэдэнд хуваагдах байрлал бүр 4-т ч хуваагдана. Хоёрдугаарт, $A = \{6, 10\}$ үед 1-үүдийн хоорондын зай 4, 2, 6, 2, … гэж жигд бус өөрчлөгдөнө. Ритмтэй мөрийн 1-үүд яг ижил зайтай байх ёстой.



Зураг 2.1. Мөрийн эхний 18 тэмдэгт. Улбар шар хүрээ нь үеийн нэг давталтыг, тэр дотор яг нэг 1 байгааг харуулна.

2.3 Гол теорем

Үлдээсэн элементүүдийн олонлогийг A гэе. Бүх элементийг хасвал мөрөнд 1 огт байхгүй тул ритмтэй биш. Иймд A хоосон биш. Цаашид тооны **үржвэр** гэж тухайн тоонд хуваагдах эерэг бүхэл тоог хэлнэ.

> **Теорем 2.1.** A хоосон биш, $m = \min A$ гэе. Мөр ритмтэй байх зайлшгүй бөгөөд хүрэлцээтэй нөхцөл нь A-ийн бүх элемент m-д хуваагдах явдал юм.

Баталгаа. **Хүрэлцээтэй.** Бүх элемент m-д хуваагддаг гэе. m өөрөө бүх үржвэрээ 1 болгох бөгөөд бусад элементийн үржвэр ч m-ийн үржвэр хэвээр. Тэгэхээр 1-үүд яг m-ийн үржвэрүүд дээр байна, $k = m$ нь үе болно. m-ээс бага k үе байж чадахгүй: тэгвэл $s_{m-k} = s_m = 1$ болох ёстой атал $m - k < m$ байрлалд 0 байна. Иймд $p = m$, эхний p тэмдэгтэд зөвхөн s_m л 1 байна.

Зайлшгүй. Мөр ритмтэй, үе нь p гэе. m-ээс бага байрлал A-ийн аль ч элементэд хуваагдахгүй тул хамгийн эхний 1 нь m байрлалд байна. Эхний p тэмдэгтэд 1 байгаа тул $m \le p$, мөн тэнд байгаа цорын ганц 1 нь s_m. Үеийн чанараар 1-үүд яг m, $m + p$, $m + 2p$, … байрлалуудад байрлана. $2m$ байрлал нь m-ийн үржвэр учир 1, тиймээс ямар нэг $c \ge 1$-ийн хувьд $2m = m + cp$, эндээс $m = cp \ge p$. $m \le p$-тэй нийлүүлбэл $m = p$. Тэгэхээр 1-үүд яг m-ийн үржвэрүүд дээр байна. A-ийн элемент a бүрт $s_a = 1$ учир $m \mid a$. (Энд гарсан бүх байрлал $2 \cdot 10^9$-аас бага тул мөрийн уртад багтана.) ■

> **Мөрдлөгөө 2.2.** Хариу нь $n - \max_{p \in a} |\{j : p \mid a_j\}|$.

Баталгаа. Дурын зөв A олонлогийн хамгийн бага элементийг p гэвэл Теорем 2.1-ээр A-ийн бүх элемент p-д хуваагдана, иймд $|A|$ нь p-д хуваагдах элементийн тооноос хэтрэхгүй. Эсрэгээр, a-ийн дурын p утгын хувьд p-д хуваагдах бүх элементийг үлдээвэл хамгийн бага нь p өөрөө байх тул зөв олонлог болно. Иймд үлдээж болох хамгийн олон элементийн тоо нь p бүрээр тоолсон утгуудын хамгийн их нь. ■

2.4 Алгоритм ба жишээ

1. Элемент бүрийг ээлжлэн p гэж сонгоно.
2. p-д үлдэгдэлгүй хуваагдах элементийн тоог тоолно. Давхардсан утга бүр тоологдоно.
3. Тоолсон утгуудын хамгийн ихийг n-ээс хасаж хэвлэнэ.

Хүснэгт 2.2. $a = [6, 10, 20, 30]$ үед. Хамгийн их нь 3 тул хариу $4 - 3 = 1$.

p	p-д хуваагдах элементүүд	Тоо
6	6, 30	2
10	10, 20, 30	**3**
20	20	1
30	30	1

2.5 Код

Код 2.1. B.cpp, 1–24-р мөр: бүтэн шийдэл

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        vector<long long> a(n);
        for (auto &x : a) cin >> x;

        int best = 0;
        for (int i = 0; i < n; i++) {
            int cnt = 0;
            for (int j = 0; j < n; j++) {
                if (a[j] % a[i] == 0) cnt++;
            }
            best = max(best, cnt);
        }
        cout << n - best << '\n';
    }
    return 0;
}
```

- **13–20-р мөр.** Гадна давталт p = `a[i]`-г сонгоно. Дотор давталт `a[j] % a[i] == 0` нөхцлөөр хуваагдах элементийг тоолно.
- **21-р мөр.** Хамгийн олон үлдээх тоог n-ээс хасна.
- Утгууд 10^9-аас хэтрэхгүй тул `int` ч хангалттай. Энд `long long` ашигласан нь алдаа биш.

2.6 Хугацаа ба анхаарах зүйл

Давхар давталт нь нэг тестэд n^2 үлдэгдэл бодно. n-ийн нийлбэр 5000 тул хамгийн ихдээ $25 \cdot 10^6$ үйлдэл бөгөөд 2 секундэд хангалттай багтана (Хавсралт A). Санах ой $O(n)$.

> **Анхаарах**
>
> **Хамгийн бага элемент заавал оновчтой биш.** a = [6, 10, 20, 30] үед 6-г сонговол 2 элемент, 10-ыг сонговол 3 элемент үлдэнэ. Тиймээс бүх p-г шалгах хэрэгтэй.
>
> **Хамгийн их ерөнхий хуваагчаар (ХИЕХ) бодож болохгүй.** a = [4, 6] үед ХИЕХ нь 2 боловч 2 массивт байхгүй. Хоёуланг нь үлдээвэл мөр 000101… болж ритмтэй биш. Хариу нь 1.
>
> **1 массивт байвал.** p = 1 үед бүх элемент тоологдож хариу 0 болно. Бүх тэмдэгт 1, үе нь 1, эхний нэг тэмдэгтэд яг нэг 1 байгаа тул энэ нь зөв.

Дүгнэлт

Мөрийн урт хэчнээн их байсан ч түүний бүтцийг теоремоор тодорхойлж чадвал мөрийг үүсгэх шаардлагагүй болно. Бодлогын бүх хүндрэл нь хариуг энгийн тоололд хүргэх нэг теорем дээр байв.

Дасгал

2.1. a = [2, 4, 6, 9, 18, 3] үед хариу хэд вэ? Аль p оновчтой вэ?

2.2. a = [5, 5, 5] үед мөр ямар харагдах вэ? Хариу хэд вэ?

БҮЛЭГ 3

C. Big XOR

Том XOR

Сэдэв битийн үйлдэл, GF(2) дээрх шугаман алгебр **Файл** `C.cpp` **Хэрэгсэл** шугаман суурь (XOR basis)

Нөхцөл дэх $m \geq 2k$ тэнцэтгэл биш нь эхлээд сул мэт боловч үнэндээ m-ийг бараг яг тодорхойлдог. Үүнийг ойлгосны дараа бодлого нь XOR-ын шугаман тэгшитгэл болно.

3.1 Бодлого

Нэг бүхэл тоо n өгөгдөнө. $a = [1, 2, \ldots, n]$ массивын дурын элементүүдийг сонгож, сонгосон бүрийг 2 дахин ихэсгэж b массив авна. $k = a_1 \mid a_2 \mid \ldots \mid a_n$, $m = b_1 \oplus b_2 \oplus \ldots \oplus b_n$ гэж тэмдэглэе. $m \geq 2k$ байхаар сонгох элементүүдийг ол. Хариу заавал олддог бөгөөд боломжит хариунуудын аль нэгийг хэвлэхэд хангалттай.

Хязгаар: $t \leq 10^4$, $1 \leq n \leq 2 \cdot 10^5$, бүх тестийн n-ийн нийлбэр $2 \cdot 10^5$-аас хэтрэхгүй. **Гаралт:** n ширхэг тоо $d_1, \ldots, d_n \in \{0, 1\}$, $d_i = 1$ бол i-г давхарлана.

Жишээ. $n = 3$ үед $k = 3$. 2-ыг давхарлавал $b = [1, 4, 3]$, $m = 1 \oplus 4 \oplus 3 = 6 \geq 6$.

3.2 Зорилтын утгыг тодорхойлох

n-ийн хоёртын бичлэгийн битийн тоог L гэж, $W = L + 1$ гэж тэмдэглэе. Жишээ нь $n = 6$ буюу хоёртын 110 бол $L = 3$, $W = 4$.

> **Лемм 3.1.** $k = 2^L - 1$ бөгөөд m нь заавал $2^W - 1$ эсвэл $2^W - 2$ байх ёстой.

> ***Баталгаа.*** $j < L$ бүрт $2^j \leq n$ тул 2^j тоо массивт байна, иймд OR-ын доод L бит бүгд 1. Түүнээс дээш бит ямар ч элементэд байхгүй. Тэгэхээр $k = 2^L - 1$, $2k = 2^W - 2$.
>
> Нөгөө талаас $n < 2^L$ учир бүх $b_i \leq 2n < 2^W$, тэгэхээр b_i бүр W битэд багтах ба XOR нь ч 2^W-ээс бага. $2^W - 2 \leq m < 2^W$ нөхцлийг зөвхөн энэ хоёр утга хангана. ■

Өөрөөр хэлбэл m-ийн дээд L бит бүгд 1 байх ёстой, зөвхөн хамгийн бага бит нь чөлөөтэй.

3.3 Давхарлах үйлдлийг XOR-оор илэрхийлэх

Хэрэв ямар ч элементийг давхарлаагүй бол $m = \text{base}$, энд $\text{base} = 1 \oplus 2 \oplus \ldots \oplus n$. i-г давхарлахад XOR доторх i нь $2i$-ээр солигдоно. XOR-ын хувьд энэ нь $d_i = i \oplus 2i$ утгаар XOR хийхтэй ижил, учир нь $i \oplus d_i = 2i$. Давхарласан индексүүдийн олонлогийг S гэвэл:

$$m = \text{base} \oplus \left(\bigoplus_{i \in S} d_i\right)$$

Иймд зорилтын утга M-ийг сонгоод дараах тэгшитгэлийг бодно:

$$\bigoplus_{i \in S} d_i = T, \qquad T = M \oplus \text{base}$$

Хүснэгт 3.1. $n = 6$ үеийн d_i утгууд. Сүүлийн баганыг анхааралаарай: 1-ийн тоо үргэлж тэгш. Энэ нь 3.6-р хэсэгт чухал үүрэг гүйцэтгэнэ.

i	i (хоёртын)	$2i$ (хоёртын)	d_i (хоёртын)	d_i	1-ийн тоо
1	0001	0010	0011	3	2
2	0010	0100	0110	6	2
3	0011	0110	0101	5	2
4	0100	1000	1100	12	2
5	0101	1010	1111	15	4
6	0110	1100	1010	10	2

3.4 Шугаман суурь

n их тул 2^n дэд олонлогийг шалгах боломжгүй. Гэхдээ бүх d_i нь $W \le 19$ битийн тоо. XOR нь бит тус бүрт 2-ын модулиар нэмэх үйлдэл учир d_i-үүдийг вектор гэж үзэж болох бөгөөд тэдгээрийн дотроос шугаман хамааралгүй нь W-ээс олонгүй байна.

> **Хэрэгсэл · шугаман суурь**
>
> **Суурь** гэдэг нь хамгийн дээд бит нь бүгд өөр өөр байх тоонуудын олонлог юм. Дээд бит нь `bit` байх векторoo `bv[bit]`-д хадгална.
>
> **Нэмэх.** v тоог дээд битээс нь доош шалгана. Тухайн бит v-д 1 бөгөөд тэр битэд вектор байвал $v \leftarrow v \oplus$ `bv[bit]`. Вектор байхгүй бол v-г тэнд хадгалаад зогсоно. v тэг болвол шинэ мэдээлэлгүй гэсэн үг.
>
> **Шалгах.** T-г яг ижил аргаар багасгана. Хэрэв 1 бит таарсан ч вектор нь байхгүй бол T-г үүсгэх боломжгүй. Төгсгөлд нь 0 болбол ашигласан векторуудын XOR нь T-тэй тэнцүү.
>
> **Яагаад зөв вэ.** Дээд битүүд нь ялгаатай тул суурийн векторуудын дурын хоосон биш XOR-ын дээд бит нь ашигласан векторуудын дээд битүүдийн хамгийн их нь байна. Тиймээс T-ийн дээд битэд таарах векторыг заавал ашиглах ёстой бөгөөд өөр сонголт байхгүй.

Бидэнд зөвхөн хариу байгаа эсэх биш, **аль индексүүдийг** ашигласан нь хэрэгтэй. Иймд вектор бүрт түүний бүрэлдэхүүнийг битийн маскаар хадгална. Суурьт шинээр орсон i-г **эзэн** гэж нэрлэе. Эзэд нь `owners` жагсаалтад дарааллаараа бичигдэх ба `bm[bit]`-ийн j-р бит нь `owners[j]` энэ векторт орсон эсэхийг заана. Эзэд W-ээс олонгүй учир маск нь `int`-д багтана.

> **Ажиглалт 3.2.** Бүх үед $\mathrm{bv}_{\mathrm{bit}}$ нь $\mathrm{bm}_{\mathrm{bit}}$ маскаар заагдсан эздийн d утгуудын XOR-той тэнцүү. Шинэ d_i-г багасгах явцад ашигласан векторуудын маскийг `mask`-д XOR-лож явдаг учир энэ чанар хадгалагдана.

Гараар мөрдөх: $n = 3$

$n = 3$ үед $L = 2$, $W = 3$, base $= 1 \oplus 2 \oplus 3 = 0$, харин $d = (3, 6, 5)$.

Хүснэгт 3.2. Суурь бүтээх ба зорилтыг шалгах.

Алхам	Үйлдэл	Үр дүн
$i = 1$	$v = 011$, бит 1 сул	`bv[1] = 011`, эзэн: 1
$i = 2$	$v = 110$, бит 2 сул	`bv[2] = 110`, эзэн: 2
$i = 3$	$v = 101$; $\oplus$ 110 $\to$ 011; $\oplus$ 011 $\to$ 000	тэг болсон: шинэ вектор үгүй
$M = 7$	$T = 111$; $\oplus$ 110 $\to$ 001; бит 0-д вектор алга	боломжгүй
$M = 6$	$T = 110$; $\oplus$ 110 $\to$ 000	$S = \{2\}$, хариу: 0 1 0

3.5 Алгоритм ба код

1. L, W, base-г тооцно.
2. $i = 1, \ldots, n$ бүрт d_i-г суурьт нэмж, шинээр орсон вектор бүрийн эзнийг бүртгэнэ.
3. $M = 2^W - 1$, дараа нь $M = 2^W - 2$ зорилтыг суурь ашиглан шалгана.
4. Бүтсэн зорилтын маскт байгаа эздийг давхарлаж хариуг хэвлэнэ.

Код 3.1. `C.cpp`, 1–14-р мөр: оролт, L, W, base

```cpp
#include<bits/stdc++.h>
using namespace std;
int main(){
    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;

        int L = 32 - __builtin_clz(n);
        int W = L + 1;

        int base = 0;
        for (int i = 1; i <= n; i++) base ^= i;
```

`__builtin_clz(n)` нь 32 битийн тооны эхний тэгүүдийг тоолдог GCC-ийн функц тул `32 - __builtin_clz(n)` нь n-ийн битийн тоо юм. $n \geq 1$ учир аюулгүй.

Код 3.2. C.cpp, 16–37-р мөр: суурь бүтээх

```
16  vector<int> bv(W, 0), bm(W, 0);
17  vector<bool> has(W, false);
18  vector<int> owners;
19
20  for (int i = 1; i <= n; i++) {
21      int v = i ^ (2 * i);
22      int mask = 0;
23      for (int bit = W - 1; bit >= 0; bit--) {
24          if (!((v >> bit) & 1)) continue;
25          if (has[bit]) {
26              v ^= bv[bit];
27              mask ^= bm[bit];
28          } else {
29              int oid = (int)owners.size();
30              owners.push_back(i);
31              has[bit] = true;
32              bv[bit] = v;
33              bm[bit] = mask ^ (1 << oid);
34              break;
35          }
36      }
37  }
```

- **21-р мөр.** d_i-г тооцно. **22-р мөр.** `mask` нь v-д одоогоор XOR хийгдсэн векторуудын бүрэлдэхүүн.
- **25–27-р мөр.** Тухайн битэд вектор байвал v-г багасгаж, маскийг шинэчилнэ.
- **29–34-р мөр.** Сул бит олдвол i шинэ эзэн болно. Ажиглалт 3.2 хадгалагдахын тулд `mask`-д i-ийн битийг нэмнэ.

Код 3.3. C.cpp, 39–63-р мөр: хоёр зорилтыг шалгаж хариуг хэвлэх

```
39          vector<int> ans(n + 1, 0);
40          for (int M : {(1 << W) - 1, (1 << W) - 2}) {
41              int v = M ^ base;
42              int acc = 0;
43              bool ok = true;
44              for (int bit = W - 1; bit >= 0; bit--) {
45                  if (!((v >> bit) & 1)) continue;
46                  if (!has[bit]) { ok = false; break; }
47                  v ^= bv[bit];
48                  acc ^= bm[bit];
49              }
50              if (ok && v == 0) {
51                  for (int j = 0; j < (int)owners.size(); j++)
52                      if ((acc >> j) & 1) ans[owners[j]] = 1;
53                  break;
54              }
55          }
56
57          for (int i = 1; i <= n; i++) {
58              cout << ans[i];
59              cout << (i == n ? '\n' : ' ');
60          }
61      }
62      return 0;
63  }
```

- **40-р мөр.** Лемм 3.1-ийн хоёр утгыг ээлжлэн туршина.
- **44–49-р мөр.** T-г багасгаж, ашигласан векторуудын маскийг `acc`-д цуглуулна.
- **50–54-р мөр.** Амжилттай бол `acc`-д байгаа эздийг давхарлана. `acc` нь XOR хэлбэрээр цуглардаг тул хоёр удаа орсон эзэн автоматаар хасагдана.

3.6 Яагаад хариу заавал олддог вэ

Дээрх код хоёр зорилтын аль нэг нь бүтэх гэж найдаж байна. Энэ нь үргэлж биелэх үү? Тэгш сондгойн чанар үүнд хариулна.

Лемм 3.3. d_i бүрийн 1-ийн тоо тэгш. Иймд d_i-үүдийн дурын XOR-ын 1-ийн тоо ч тэгш.

Баталгаа. Хоёр тооны XOR хийхэд хоёуланд нь 1 байсан бит бүр 2-оор хасагддаг тул $\mathrm{popcount}(a \oplus b)$ ба $\mathrm{popcount}(a) + \mathrm{popcount}(b)$-ийн тэгш сондгой нь ижил. $2i$-д i-тэй ижил тооны 1 байгаа тул $\mathrm{popcount}(d_i)$ нь $2 \cdot \mathrm{popcount}(i)$-тэй тэгш сондгойн хувьд ижил буюу тэгш. Хоёр тэгш тооны XOR мөн тэгш гэдгийг дээрх чанараас шууд авна. ■

Лемм 3.4. W битийн дурын 1-ийн тоо нь тэгш T-г зөвхөн хоёрын зэрэгт индексүүдийг ашиглан гаргаж болно.

Баталгаа. $i = 2^j$ үед $d_i = 2^j \oplus 2^{j+1} = 3 \cdot 2^j$, өөрөөр хэлбэл яг j ба $j+1$ битүүд 1. $j = 0, 1, \ldots, L-1$ дарааллаар явж, T-ийн j-р бит 1 бол d_{2^j}-ээр XOR хийнэ. Энэ нь j-р битийг цэвэрлэж, $j+1$-р битийг урвуулна. Төгсгөлд $0, \ldots, L-1$ битүүд бүгд 0, зөвхөн L-р бит үлдэж болно. Лемм 3.3-ын ёсоор XOR хийхэд тэгш сондгой өөрчлөгдөхгүй тул үлдэгдлийн 1-ийн тоо тэгш, тиймээс L-р бит мөн 0. $j < L$ учир $2^j \le n$ бөгөөд иймд индекс массивт бий. ■

Теорем 3.5. Хоёр зорилтын **яг нэг** нь үргэлж бүтнэ. Тэр нь $M \oplus$ base-ийн 1-ийн тоо тэгш байх M юм.

Баталгаа. $2^W - 1$ ба $2^W - 2$ нь зөвхөн хамгийн бага битээрээ ялгаатай. Тэгэхээр тэдгээрийн base-тэй XOR-ууд мөн нэг битээр ялгаатай бөгөөд 1-ийн тооны тэгш сондгой нь эсрэг. Тэгш нь Лемм 3.4-өөр бүтнэ, сондгой нь Лемм 3.3-ын ёсоор бүтэхгүй. ■

3.7 Илүү энгийн хувилбар

Лемм 3.4-ийн баталгаа өөрөө алгоритм юм: шугаман суурь огт хэрэггүй! Зөвхөн хоёрын зэрэгт индексүүдийг давхарлаад л хариу гарна. `C.cpp` дахь ерөнхий арга ч зөв ажилладаг бөгөөд олон төрлийн бодлогод хэрэглэгддэг тул номонд үндсэн аргаар тайлбарласан. Гэхдээ ажиглалт нь кодыг хэрхэн хялбарчилж болохыг харуулах сайн жишээ.

Доорх код нь `C.cpp`-ийн 10–60-р мөрийг бүхэлд нь орлож чадна.

Код 3.4. Энгийн хувилбар: хоёрын зэрэгт индексээр шийдэх

```cpp
int L = 32 - __builtin_clz(n);
int W = L + 1;

int base = 0;
for (int i = 1; i <= n; i++) base ^= i;

int T = ((1 << W) - 1) ^ base;
if (__builtin_popcount(T) % 2 == 1) T ^= 1;

vector<int> d(n + 1, 0);
for (int j = 0; j < L; j++) {
    if ((T >> j) & 1) {
        d[1 << j] = 1;
        T ^= 3 << j;
    }
}

for (int i = 1; i <= n; i++) cout << d[i] << (i == n ? '\n' : ' ');
```

`__builtin_popcount(T)` нь T-ийн 1 битийн тоог буцаадаг GCC-ийн функц. 7–8-р мөрд $M = 2^W - 1$-ээс эхэлж, T-ийн 1-ийн тоо сондгой бол хамгийн бага битийг нь урвуулж $M = 2^W - 2$ рүү шилжинэ. Зураг 3.1-д $n = 6$ үеийг мөрдөв: base = 7, $T = 15 \oplus 7 = 8$-ийн 1-ийн тоо сондгой тул $T = 9$ болно.



Зураг 3.1. T-г доод битээс нь цэвэрлэх нь. Улбар шар нүд нь тухайн алхамд урвуулсан хоёр бит. Хариу $S = \{1, 2, 4\}$, $b = [2, 4, 3, 8, 5, 6]$, $m = 14 = 2k$.

3.8 Хугацаа ба анхаарах зүйл

`C.cpp` нь d_i бүрийг W-ээс олонгүй алхмаар багасгах тул хугацаа $O(n \log n)$, санах ой $O(n)$. Энгийн хувилбар нь $O(n)$.

> **Анхаарах**
>
> **Битийн тоо нь L биш, $W = L + 1$.** $2i$ нь L-р битэд хүрдэг тул суурийн массив W хэмжээтэй байх ёстой.
>
> **Өөрчлөлт нь $2i$ биш, $i \oplus 2i$.** Давхарлах нь i-г XOR-оос хасаад $2i$-г нэмж буй хэрэг.
>
> **m заавал $2k$-тай тэнцэх албагүй.** $2^W - 1$ утга ч нөхцөлийг хангана. Хоёр боломжийг хоёуланг нь шалгах хэрэгтэй.

Дүгнэлт

Тэнцэтгэл биш нь хязгаартай бол хариу хэдхэн утга л авч болдог эсэхийг шалгаарай. XOR-ын сонголтын бодлогыг шугаман систем гэж харж, тэгш сондгой шиг хувиршгүй чанарыг хайх нь шийдлийг мэдэгдэхүйц хялбарчилдаг.

Дасгал

3.1. $n = 5$ үед энгийн хувилбарыг гараар мөрд. Аль индексүүд давхарлагдах вэ?

3.2. $M = 2^W - 1$ ба $M = 2^W - 2$ хоёр зорилт яагаад зэрэг бүтэж чадахгүй вэ?

БҮЛЭГ 4

D. Black, Gray and White

Хар, саарал, цагаан

Сэдэв мод, дэд модны нийлбэр, санамсаргүй хэш **Файл** `D.cpp`
Хэрэгсэл BFS дараалал, дэд модны нийлбэр, хэш

k-ийн $2n$ утгын хувьд мод дээр зам будах нь удаан мэт санагдана. Гэтэл хар оройн тоог хамгийн их болгох шаардлага k-ийн сонголтыг гурав хүртэл багасгадаг.

4.1 Бодлого

n оройтой мод өгөгдөнө, бүх орой эхэндээ цагаан. $1 \le k \le 2n$ бүхэл тоо сонгоно. Дараа нь $i = 1, 2, \ldots, n$ дарааллаар, хэрэв $k - i$ дугаартай орой байгаа бөгөөд $k - i \neq i$ бол:

- i ба $k - i$ оройг хар болгоно (өмнөх өнгөнөөс үл хамааран);
- тэдгээрийн хоорондох зам дээрх **цагаан** оройнуудыг саарал болгоно.

Эхлээд хар оройн тоог, түүний дараа саарал оройн тоог хамгийн их болгох k-г ол. Хэд хэдэн хариу байвал дурын нэгийг хэвлэнэ. **Хязгаар:** $3 \le n \le 3 \cdot 10^5$, n-ийн нийлбэр $3 \cdot 10^5$-аас хэтрэхгүй.

4.2 Эцсийн өнгө

k тогтмол үед $x + y = k$, $x \neq y$ байх хоёр оройг **хос** гэе. Орой бүр хамгийн ихдээ нэг хост ордог, учир нь x-ийн хос зөвхөн $k - x$ байж болно. Хост орсон оройг **хосолсон** гэж нэрлэе.

> **Лемм 4.1.** Эцсийн өнгө үйлдлийн дарааллаас хамаарахгүй: хосолсон орой хар; хосолоогүй боловч ямар нэг хосын зам дээр байгаа орой саарал; бусад нь цагаан.

> ***Баталгаа.*** Хар болсон орой дахин өнгө солихгүй, учир нь зам дээр зөвхөн цагаан орой л саарал болдог. Хосолсон орой эцэст нь заавал хар болно. Хосолоогүй оройн хувьд ганц боломжит өөрчлөлт нь замд орж цагаанаас саарал болох бөгөөд энэ нь ямар нэг хосын замд орвол заавал тохиолдоно. ■

Тиймээс хар оройн тоо бол хосолсон оройн тоо, саарал оройн тоо бол замд орсон хосолоогүй оройн тоо юм.

4.3 Хар оройн тоо ба нэр дэвшигч k

x хосолсон байхын тулд $1 \le k - x \le n$ байх ёстой. Энэ нь $k \le n + 1$ үед $x \in [1, k - 1]$, $k \ge n + 1$ үед $x \in [k - n, n]$ гэсэн үг. Мөн k тэгш бол $x = k / 2$ нь өөртэйгөө хос болох тул тооцохгүй:

$$\text{black}(k) = \min(k - 1,\ 2n - k + 1) - (k \text{ тэгш бол } 1, \text{ сондгой бол } 0)$$

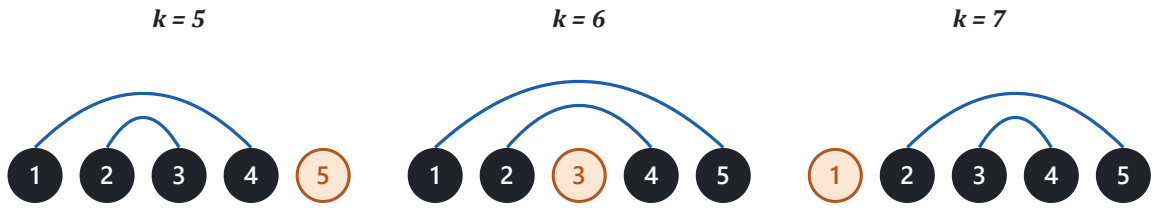
Хүснэгт 4.1. black(k)-ийн утгууд. Хамгийн их утгыг тодруулав.

	$k=1$	$k=2$	$k=3$	$k=4$	$k=5$	$k=6$	$k=7$	$k=8$	$k=9$	$k=10$	$k=11$	$k=12$
$n=5$	0	0	2	2	**4**	**4**	**4**	2	2	0	–	–
$n=6$	0	0	2	2	4	4	**6**	4	4	2	2	0

Теорем 4.2. n тэгш бол зөвхөн $k = n + 1$ үед бүх орой хар болно. n сондгой бол хар оройн хамгийн их тоо $n - 1$ бөгөөд яг гурван k үүнд хүрнэ. Эдгээр k тус бүрд яг нэг орой хосолоогүй үлдэнэ (доорх хүснэгт).

k	Хосолоогүй орой v	Яагаад
n	n	$k - n = 0$ гэсэн орой байхгүй
$n+1$	$(n+1)/2$	k тэгш, энэ орой өөртэйгөө хос
$n+2$	1	$k - 1 = n + 1$ гэсэн орой байхгүй

Баталгаа. Томьёог шалгахад хангалттай. n тэгш бол $k = n + 1$ сондгой, black $= n$. Бусад k-д $\min(k-1, 2n-k+1) \le n - 1$ тул n-д хүрэхгүй. n сондгой бол $k = n + 1$ тэгш тул black $= n - 1$; $k = n$ ба $k = n + 2$ сондгой тул мөн $n - 1$. $k = n - 1$ ба $k = n + 3$ тэгш тул $n - 3$, цаашдах k-д $\min(k-1, 2n-k+1) \le n - 3$ тул бага. ■

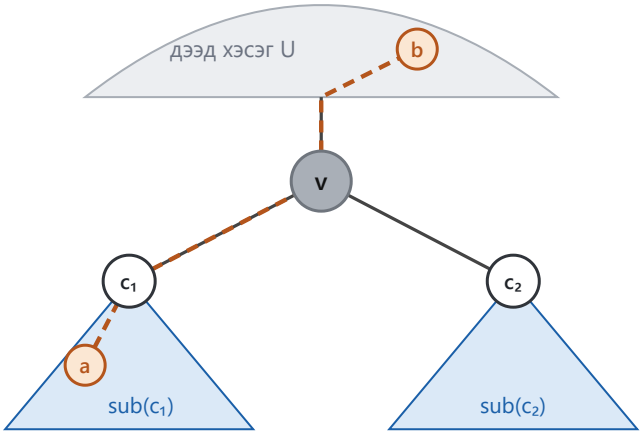


Зураг 4.1. $n = 5$ үеийн гурван нэр дэвшигч k. Цэнхэр нум нь хосыг, улбар шар орой нь хосолоогүй оройг заана.

4.4 Саарал орой

Теорем 4.2-ын ачаар саарал оройн тухай бодох нь маш хялбар болно. n тэгш бол хосолоогүй орой алга тул хариу шууд $k = n + 1$. n сондгой бол хосолоогүй ганц орой v байгаа тул саарал орой 0 эсвэл 1 ширхэг. Асуулт нэг л зүйл дээр тогтоно: **v ямар нэг хосын зам дээр байна уу?**

v-г модноос авч хаявал мод хэд хэдэн хэсэгт (компонентод) задарна. a, b-ийн хоорондох зам v-г дайрах нь a, b өөр компонентод байхтай ижил. Модыг 1-р оройгоор үндэслэвэл эдгээр компонент нь v-ийн хүүхэд c бүрийн дэд мод sub(c) ба үлдсэн дээд хэсэг U юм (Зураг 4.2).



Зураг 4.2. *v*-г хасахад үүсэх компонентууд. *a*, *b* хос өөр компонентод байгаа тул зам нь заавал *v*-г дайрна.

Хэрэгсэл · дэд модны нийлбэр ба хэш

Орой бүрт санамсаргүй 64 битийн жин w_x өгье. Хосолсон x-ийн хос нь y бол $\mathrm{val}(x) = w_x - w_y$, хосолоогүй бол $\mathrm{val}(x) = 0$ гэж тодорхойлъё. Тооцоог 2^{64}-ийн модулиар хийнэ (`unsigned long long` -ийн халилт яг ийм). S оройн олонлогийн хувьд $D(S)$ нь val утгуудын нийлбэр.

Хос бүрийн хувьд $\mathrm{val}(x) + \mathrm{val}(y) = 0$ тул S дотор хоёр үзүүр нь хамт орсон хос нийлбэрт нөлөөлөхгүй. Нэг үзүүр нь S дотор, нөгөө нь гадна байх хосыг **таслагдсан** гэж нэрлэе. Нийлбэрт зөвхөн ийм хосууд үлдэнэ.

Лемм 4.3. S ямар ч хосыг таслахгүй бол $D(S) = 0$. Таслагдсан хос байвал $D(S) = 0$ байх магадлал 2^{-64}.

Баталгаа. Эхний хэсгийг дээр харлаа. Таслагдсан хос байвал $D(S)$-д ямар нэг w_a яг нэг удаа $+1$ эсвэл -1 коэффициенттэй орно. Бусад жинг тогтоосон үед w_a жигд санамсаргүй учир $D(S)$ ч жигд тархсан. Иймд тэгтэй тэнцэх магадлал 2^{-64}. ■

Лемм 4.4. Хосолоогүй v нь зам дээр байх нь v-ийн ямар нэг хүүхэд c-д $D(\mathrm{sub}(c)) \neq 0$ байхтай ижил (хэшийн алдааг тооцохгүй).

Баталгаа. $D(\mathrm{sub}(c)) \neq 0$ бол $\mathrm{sub}(c)$ нэг хосыг таслаж байна. Энэ хосын нөгөө үзүүр нь $\mathrm{sub}(c)$-ийн гадна байх тул зам c–v ирмэгээр гарч v-г дайрна.

Эсрэгээр, бүх хүүхдэд $D(\mathrm{sub}(c)) = 0$ гэе. Бүх оройн нийлбэр 0 ба $\mathrm{val}(v) = 0$ тул $D(U) = -\Sigma\, D(\mathrm{sub}(c)) = 0$. Тэгэхээр аль ч компонент хос таслахгүй, v замд орохгүй. ■

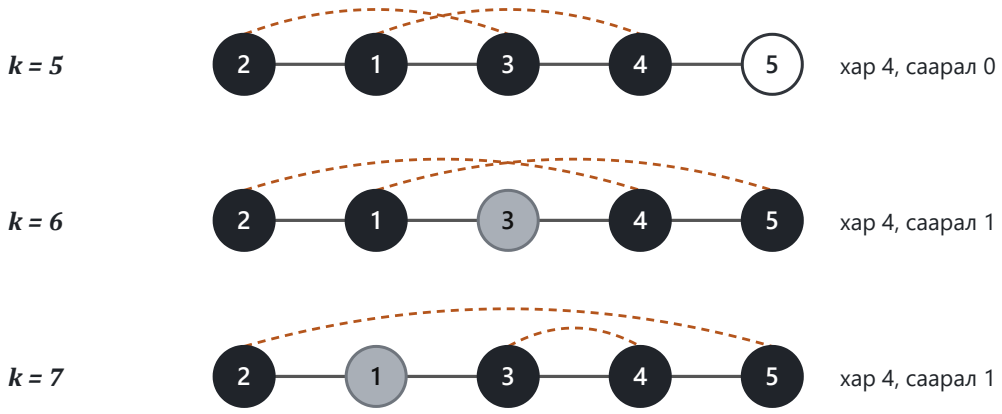
4.5 Алгоритм

1. Модыг уншаад 1-р ороогоос BFS хийж `par` (эцэг) ба `order` (нээгдсэн дараалал)-ыг олно.
2. Орой бүрт санамсаргүй w_x оноож, бүх k-д $\mathrm{black}(k)$-г тооцно.

3. black(k) нь хамгийн их байх k бүрт: val утгуудыг бичээд `order`-ийг ардаас нь гүйж дэд модны нийлбэрийг цуглуулна. Хүүхдийн нийлбэр тэг биш бол эцгийг `flag`-аар тэмдэглэнэ.
4. Хосолоогүй бөгөөд тэмдэглэгдсэн оройн тоо нь саарал оройн тоо. Хамгийн их утгатай k-г хэвлэнэ.

4.6 Жишээгээр мөрдөх

n = 5, ирмэгүүд 1–2, 1–3, 3–4, 4–5 буюу 2–1–3–4–5 зам (Зураг 4.3). k = 5 үед хосолоогүй 5-р орой замын төгсгөлд байрлах тул ямар ч хосын замд орохгүй, саарал 0. k = 6 үед 3-р орой 1–5 хосын зам дээр, k = 7 үед 1-р орой 2–5 хосын зам дээр байна. Код хамгийн түрүүнд сайжруулалт өгсөн k = 6-г хэвлэнэ.



Зураг 4.3. Жишээ модны эцсийн өнгө. Улбар шар тасархай нум нь хосуудыг, хар дугуй нь хосолсон, саарал дугуй нь замд орсон хосолоогүй оройг заана. Цагаан дугуй нь ямар ч замд ороогүй.

4.7 Код

Код 4.1. D.cpp, 1–20-р мөр: оролт ба модыг хадгалах

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (t--) {
        int n;
        cin >> n;
        vector<vector<int>> adj(n + 1);
        for (int e = 0; e < n - 1; e++) {
            int u, v;
            cin >> u >> v;
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
```

5–6-р мөр нь оролт гаралтыг хурдасгах түгээмэл бичиглэл. `mt19937_64` нь 64 битийн санамсаргүй тоо үүсгэгч. Үрийг цагаар сонгосон тул кодыг урьдчилан мэдсэн ч эвдэх тест бэлтгэх боломжгүй.

Код 4.2. D.cpp, 22–35-р мөр: BFS: par ба order

```cpp
vector<int> par(n + 1, 0), order;
vector<char> vis(n + 1, 0);
order.push_back(1);
vis[1] = 1;
for (size_t idx = 0; idx < order.size(); idx++) {
    int u = order[idx];
    for (int v : adj[u]) {
        if (!vis[v]) {
            vis[v] = 1;
            par[v] = u;
            order.push_back(v);
        }
    }
}
```

BFS-ийг рекурсгүй бичсэн шалтгаан нь $n = 3 \cdot 10^5$ үед гүн модон дээр рекурс стекийг дүүргэж болох явдал юм. `order`-т эцэг нь хүүхдээсээ үргэлж өмнө ордог тул ардаас нь явахад хүүхдүүд эцгээсээ түрүүлж боловсруулагдана.

Код 4.3. D.cpp, 37–47-р мөр: жин ба хар оройн тоо

```
vector<unsigned long long> w(n + 1);
for (int x = 1; x <= n; x++) w[x] = rng();

auto blackOf = [&](int k) -> long long {
    int lo = max(1, k - n), hi = min(n, k - 1);
    long long cnt = hi >= lo ? hi - lo + 1 : 0;
    if (k % 2 == 0 && k / 2 >= lo && k / 2 <= hi) cnt--;
    return cnt;
};
long long bestBlack = 0;
for (int k = 1; k <= 2 * n; k++) bestBlack = max(bestBlack, blackOf(k));
```

`blackOf` нь Теорем 4.2-ын томьёог ерөнхий хэлбэрээр тооцно: `lo`, `hi` нь x-ийн мужийн хоёр зах. Энэ функц $O(1)$ тул бүх k-г шалгахад нийт $O(n)$.

Код 4.4. D.cpp, 49–62-р мөр: нэр дэвшигч бүрт val утгуудыг бичих

```
int bestK = -1;
long long bestGray = -1;
for (int k = 1; k <= 2 * n; k++) {
    if (blackOf(k) != bestBlack) continue;

    vector<unsigned long long> sub(n + 1, 0);
    vector<char> paired(n + 1, 0), flag(n + 1, 0);
    for (int x = 1; x <= n; x++) {
        int j = k - x;
        if (j >= 1 && j <= n && j != x) {
            paired[x] = 1;
            sub[x] = w[x] - w[j];
        }
    }
```

- **52-р мөр.** Теорем 4.2-ын ёсоор энэ шалгуурыг гурваас олон k давахгүй, тиймээс доорх $O(n)$ хэсэг хамгийн ихдээ гурван удаа ажиллана.
- **58–61-р мөр.** Хос бүрт $\mathrm{val}(x) = w_x - w_y$ бичнэ. `sub[x]` эхэндээ зөвхөн $\mathrm{val}(x)$.

Код 4.5. D.cpp, 63–81-р мөр: дэд модны нийлбэр, саарал оройн тоо, хариу

```
            for (int idx = n - 1; idx >= 1; idx--) {
                int x = order[idx];
                int p = par[x];
                if (sub[x] != 0) flag[p] = 1;
                sub[p] += sub[x];
            }
            long long gray = 0;
            for (int x = 1; x <= n; x++)
                if (!paired[x] && flag[x]) gray++;

            if (gray > bestGray) {
                bestGray = gray;
                bestK = k;
            }
        }
        cout << bestK << '\n';
    }
    return 0;
}
```

- **63–68-р мөр.** Хүүхдийн нийлбэрийг эцэгт нэмнэ. Тэг биш бол Лемм 4.4-ийн ёсоор эцэг нь зам дээр байна.
- **69–71-р мөр.** Хосолоогүй бөгөөд тэмдэглэгдсэн оройг тоолно. Энэ нь саарал оройн тоо.
- **73–76-р мөр.** Зөвхөн хатуу сайжрахад шинэчилнэ. Тиймээс тэнцэх үед хамгийн түрүүнд олдсон k үлдэнэ.

4.8 Хэшгүй хувилбар

Теорем 4.2-ын дараа санамсаргүй тоо заавал хэрэггүй: компонент бүрийг нэг дугаараар будаад, хоёр үзүүр нь өөр дугаартай хос байгаа эсэхийг шалгахад хангалттай. `D.cpp` нь аль ч k-д ажилладаг ерөнхий арга тул хэшийг ашигласан, харин дараах хувилбар нь санамсаргүй тоо ашиглахгүй, уншихад ч хялбар. Энэ код `D.cpp`-ийн 22–78-р мөрийг бүхэлд нь орлож чадна.

Код 4.6. Хэшгүй хувилбар: компонентоор будах

```
int answer = n + 1;
if (n % 2 == 1) {
    vector<pair<int, int>> cand = {{n, n}, {n + 1, (n + 1) / 2}, {n + 2, 1}};
    for (auto [k, v] : cand) {
        vector<int> comp(n + 1, 0);
        comp[v] = -1;
        for (int c : adj[v]) {
            vector<int> st = {c};
            comp[c] = c;
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (int w : adj[u]) {
                    if (comp[w] == 0) {
                        comp[w] = c;
                        st.push_back(w);
                    }
                }
            }
        }
        bool gray = false;
        for (int x = 1; x <= n; x++)
            if (x != v && comp[x] != comp[k - x]) gray = true;
        if (gray) {
            answer = k;
            break;
        }
    }
}
cout << answer << '\n';
```

`comp[v] = -1` нь будалт v-г давж гарахгүйг баталгаажуулна. $x \neq v$ бүрийн хос нь Теорем 4.2-ын ёсоор заавал байх тул `comp[k - x]` хүчинтэй.

4.9 Хугацаа ба анхаарах зүйл

Модыг унших ба BFS $O(n)$, нэр дэвшигч бүрт $O(n)$, тэдгээр нь гурваас олонгүй. Нийт $O(n)$ хугацаа, $O(n)$ санах ой.

Анхаарах

Хэшийн алдааны магадлал. Нэг шалгалтад 2^{-64}, нийт $3n \approx 10^6$ шалгалтад $5 \cdot 10^{-14}$-өөс бага. Практикт энэ нь нөлөөгүй.

Рекурс. Зам хэлбэрийн модон дээр гүн нь $3 \cdot 10^5$ хүрэх тул Windows дээрх анхдагч стекийг дүүргэж болно.

***k* тэгш үеийн тэнцүү хос.** $x = k / 2$ үед $k - x = x$ тул энэ оройг хосолсонд тооцохгүй. Үүнийг мартвал n сондгой үед $k = n + 1$ буруу үнэлэгдэнэ.

Дүгнэлт

Сонголт олон мэт харагдаж байвал эхлээд гол шалгуурыг томьёолоорой. Энд хар оройн тоо хамгийн их байх шаардлага хайх орон зайг $2n$-ээс гурав болгож, бодлогын үлдсэн хэсгийг ганц оройн тухай асуулт болгов.

Дасгал

4.1. $n = 8$ үед модны хэлбэрээс үл хамааран хариу юу вэ? Яагаад?

4.2. $n = 5$, бүх ирмэг 5-р оройд холбогдсон од хэлбэрийн мод өгөгдөв. $k = 5, 6, 7$ тус бүрд хар ба саарал оройн тоог ол.

БҮЛЭГ 5

E. Partition of a Binary String

Хоёртын мөрийн хуваалт

Сэдэв комбинаторик, Беллийн тоо **Файл** `E.cpp` **Хэрэгсэл** Беллийн гурвалжин

Анх харахад хуваалтын бүтэц төвөгтэй мэт. Гэвч нөхцөлийн зөвшөөрөх холимог олонлог маш цөөн хэлбэртэй бөгөөд оновчтой хуваалтад аль байрлал холимог олонлогт орох нь урьдчилан тодорхой болдог.

5.1 Бодлого

0 ба 1-ээс бүрдэх n урт s мөр өгөгдөнө. 1, …, n тоонуудыг хоосон биш олонлогуудад хуваана. Нэг олонлог дотор дурын ялгаатай x, y-ийн хувьд $s_x = s_y$ эсвэл $|x - y| = 1$ байх ёстой.

M олонлогийн өртөг нь $|M| \cdot f(M)$, энд $f(M)$ нь M-ийн байрлалуудад гарах ялгаатай тэмдэгтийн тоо (1 эсвэл 2). Хуваалтын өртөг нь олонлогуудын өртгийн нийлбэр. **Хамгийн их өртгийг**, мөн **ийм өртөгтэй хуваалтын тоог** 998244353-ын модулиар ол. Олонлогуудын болон доторх элементүүдийн дараалал хамаагүй.

Хязгаар: $t \le 500$, $n \le 5000$. Тэмцээний зарлалаар n-ийн нийлбэрт хязгаар **байхгүй**, өөрөөр хэлбэл хамгийн муу үед 500 удаа $n = 5000$ тест ирж болно.

Хүснэгт 5.1. Жишээ тестүүд.

s	Хамгийн их өртөг	Хуваалтын тоо
`10001000`	13	5
`000111`	8	4
`01010101`	16	4

5.2 Жишээгээр ойлгох

s = 10001000 үед {1, 2}, {3, 8}, {4, 5, 6}, {7} хуваалтыг авъя. {1, 2}-т тэмдэгтүүд 1, 0 ба байрлал нь зэргэлдээ, өртөг $2 \cdot 2 = 4$. {3, 8}-д хоёр нь 0, өртөг $2 \cdot 1 = 2$. {4, 5, 6}-д 0, 1, 0: 4 ба 6 нь ижил тэмдэгттэй, бусад нь зэргэлдээ тул зөв, өртөг $3 \cdot 2 = 6$. {7}-ийн өртөг 1. Нийт 13.

Тэмдэгт холилдсон олонлог өртгийг хоёр дахин нэмдэг нь харагдаж байна. Тиймээс асуулт нь: ямар олонлог холимог байж болох вэ?

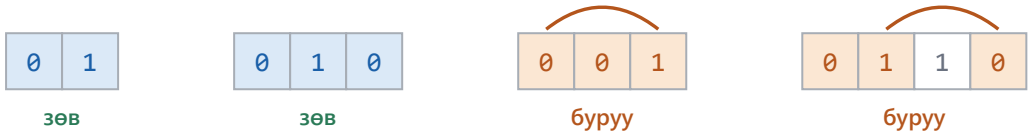
5.3 Холимог олонлог ямар байх вэ

Хоёр төрлийн тэмдэгт агуулсан олонлогийг **холимог**, нэг төрлийн тэмдэгттэйг **нэгэн төрлийн** гэж нэрлэе. Нэгэн төрлийн олонлогт нөхцөл шууд биелэх тул тэдгээр нь дурын байрлалуудаас тогтож болно.

> **Лемм 5.1.** Холимог олонлог нь зөвхөн дараах хоёр хэлбэртэй: зэргэлдээ хоёр байрлал ($s_i \ne s_{i+1}$), эсвэл зэргэлдээ гурван байрлал ($s_i = s_{i+2} \ne s_{i+1}$).

Баталгаа. M холимог, Z нь 0-тэй байрлалууд, O нь 1-тэй байрлалууд гэе. $z \in Z$, $o \in O$ бүрийн тэмдэгт өөр тул $|z - o| = 1$ байх ёстой. Тэгэхээр $O \subseteq \{z - 1, z + 1\}$, иймд $|O| \le 2$. Мөн адил $|Z| \le 2$.

Хэрэв $|Z| = 2$, $Z = \{z_1, z_2\}$ бол o бүр хоёуланд нь зэргэлдээ байх ёстой. Энэ нь зөвхөн $z_2 = z_1 + 2$, $o = z_1 + 1$ үед боломжтой, тиймээс $|O| = 1$. Тэгш хэмээр $|O| = 2$ бол $|Z| = 1$. Иймд хэмжээ нь (1, 1), (1, 2) эсвэл (2, 1) бөгөөд байрлалууд нь дараалсан байна. ■



Зураг 5.1. Сонгосон байрлалуудыг өнгөөр ялгав. Улбар шар нум нь тэмдэгт нь өөр боловч зэргэлдээ биш, нөхцөлийг зөрчсөн хосыг заана.

5.4 Өртгийг хялбарчлах

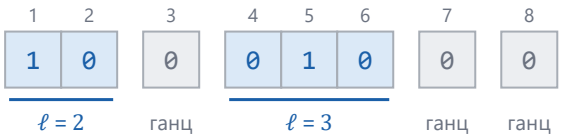
Нэгэн төрлийн олонлогийн өртөг $|M|$, холимог олонлогийнх $2|M|$. Бүх олонлогийн хэмжээний нийлбэр n тул:

$$\text{өртөг} = n + X,\ X = \text{холимог олонлогуудын хэмжээний нийлбэр}$$

Иймд зорилго нь **аль болох олон байрлалыг холимог олонлогт оруулах** явдал. Нэгэн төрлийн хэсгийг яаж хуваах нь өртөгт огт нөлөөлөхгүй: энэ нь хожим тоололд чухал болно.

5.5 Ээлжилсэн хэсгүүд

Тодорхойлолт 5.2. Мөрийг $s_i = s_{i+1}$ байх газар бүрт тасалбал хэд хэдэн хэсэг гарна. Эдгээрийг **ээлжилсэн хэсэг** гэнэ. Хэсэг бүрийн дотор хөрш тэмдэгтүүд үргэлж ялгаатай. Урт нь 1 байх хэсгийг **ганц** хэсэг гэе.



Зураг 5.2. s = 10001000-ийн ээлжилсэн хэсгүүд.

Лемм 5.3. Холимог олонлог нэг ээлжилсэн хэсгийн дотор бүтнээрээ багтана. Урт нь $\ell \ge 2$ хэсгийн дараалсан 2 эсвэл 3 байрлал бүр нь зөв холимог олонлог. Ганц хэсгийн байрлал холимог олонлогт орж чадахгүй.

Баталгаа. Лемм 5.1-ээр холимог олонлогийн хөрш байрлалуудын тэмдэгт ялгаатай тул дотор нь тасралт байхгүй. Ээлжилсэн хэсэгт дараалсан хоёр байрлалын тэмдэгт ялгаатай, дараалсан гурвын хоёр зах нь ижил, дунд нь өөр байна. Ганц хэсгийн байрлалын хөршүүд өөртэй нь ижил тэмдэгттэй (эс бөгөөс хэсэг 1-ээс урт байх байсан) тул холимог хос үүсэхгүй. ■

Теорем 5.4. Хамгийн их өртөг нь $n + \Sigma_{\ell \geq 2}\, \ell$, энд нийлбэрийг урт нь 2-оос багагүй бүх ээлжилсэн хэсгийн уртуудаар авна. Оновчтой хуваалтад ийм хэсгийн байрлал бүр холимог олонлогт орох ба ганц хэсгийн байрлалууд нэгэн төрлийн олонлогт орно.

Баталгаа. Ганц хэсгийн байрлал Лемм 5.3-аар X-д орохгүй, тиймээс $X \leq \Sigma_{\ell \geq 2}\, \ell$. Энэ хязгаарт хүрнэ: 2-оос багагүй дурын бүхэл тоог 2 ба 3-ын нийлбэрээр бичиж болох тул хэсэг бүрийг дараалсан 2 ба 3 урт хэсгүүдэд бүрэн хуваах болно. Иймд тэнцэл биелэхэд ийм хэсгийн бүх байрлал холимог олонлогт орсон байх ёстой. ■

Хамгийн анхаарал татах үр дүн нь: **аль байрлал холимог олонлогт орох нь урьдчилан тодорхой**. Энэ нь тоололд шийдвэрлэх ач холбогдолтой, учир нь холимог олонлогт орохгүй байрлалууд сонголтоос хамаарахгүй.

5.6 Оновчтой хуваалтыг тоолох

Теорем 5.4-ийн дагуу оновчтой хуваалтыг хоёр бие даасан сонголтоор тодорхойлно.

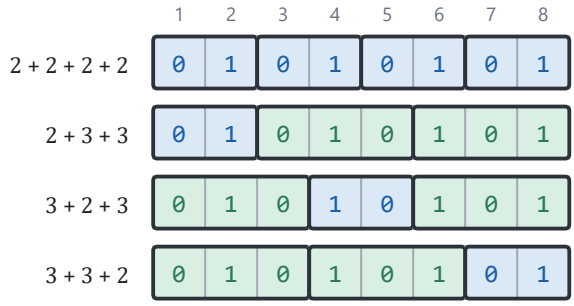
1. Ээлжилсэн хэсгүүдийг хуваах

Урт нь ℓ хэсгийг дараалсан 2 ба 3 урт хэсгүүдэд хуваах аргын тоог $c(\ell)$ гэе. Сүүлчийн хэсгийн уртаар салгавал:

$$c(0) = 1,\; c(1) = 0,\; c(\ell) = c(\ell - 2) + c(\ell - 3)$$

Хүснэгт 5.2. $c(\ell)$-ийн эхний утгууд.

ℓ	0	1	2	3	4	5	6	7	8	9	10	11	12
$c(\ell)$	1	0	1	1	1	2	2	3	4	5	7	9	12



Зураг 5.3. $\ell = 8$ үеийн $c(8) = 4$ хуваалт. Цэнхэр нь хоёр, ногоон нь гурван элементтэй холимог олонлог.

2. Ганц хэсгүүдийг хуваах

Ганц хэсгийн байрлалуудаас z нь 0, o нь 1 тэмдэгттэй гэе. Тэд зөвхөн өөрийн тэмдэгттэй нэгэн төрлийн олонлогт орох ба ямар ч хуваалт өртгийг өөрчлөхгүй. Тэгэхээр 0-үүдийг дурын аргаар, 1-үүдийг дурын аргаар хуваана. r элементтэй олонлогийг хуваах аргын тоог **Беллийн тоо** $\mathrm{Bell}(r)$ гэдэг.

Теорем 5.5. Хамгийн их өртөгтэй хуваалтын тоо нь $\mathrm{Bell}(z) \cdot \mathrm{Bell}(o) \cdot \Pi_{\ell \geq 2}\, c(\ell)$.

> ***Баталгаа.*** Холимог олонлогууд нь хуваалтаас шууд тодорхойлогдох тул тэдгээрийн өөр сонголт нь өөр хуваалт өгнө. Хэсгүүд хоорондоо болон ганц байрлалуудаас хамааралгүй сонгогдох тул аргын тоонууд үржигдэнэ. ■

5.7 Беллийн тоог тооцох

> **Хэрэгсэл · Беллийн гурвалжин**
>
> Мөр бүрийн эхний тоо нь өмнөх мөрийн **сүүлчийн** тоо. Дараагийн тоо бүр нь зүүн хөрш ба тэр хөршийн яг дээрх тооны нийлбэр. Жишээ нь 3-р мөрт 7 = 5 + 2, 10 = 7 + 3, 15 = 10 + 5. r-р мөрийн (0-ээс тоолно) эхний тоо нь Bell(r).
>
> N хүртэлх утгыг $O(N^2)$ үйлдлээр бодох ба зөвхөн сүүлчийн мөрийг хадгалахад хангалттай.

Хүснэгт 5.3. Гурвалжны эхний мөрүүд. Тодруулсан тоонууд нь Bell(0), …, Bell(4).

мөр					
0	**1**				
1	**1**	2			
2	**2**	3	5		
3	**5**	7	10	15	
4	**15**	20	27	37	52

Энэ гурвалжны томьёог энд батлахгүй. Гэхдээ үр дүнг бүх $N \le 5000$-д Стирлингийн тоогоор тодорхойлсон $\mathrm{Bell}(N) = \Sigma_k S(N, k)$ утгатай модулиар харьцуулж шалгасан.

5.8 Алгоритм ба жишээ

1. Програм эхлэхэд `bell` ба `ways` (буюу c) хүснэгтийг нэг удаа бодно.
2. Тест бүрт мөрийг зүүнээс баруун тийш гүйж, ээлжилсэн хэсэг бүрийн урт ℓ-ийг олно.
3. $\ell = 1$ бол тухайн тэмдэгтийн ганц тоолуурыг нэмнэ. Эс бөгөөс X-д ℓ-ийг нэмж, үржвэрийг $c(\ell)$-ээр үржүүлнэ.
4. $n + X$ ба $\mathrm{Bell}(z) \cdot \mathrm{Bell}(o) \cdot \Pi\, c(\ell)$-ийг хэвлэнэ.

Хүснэгт 5.4. Жишээ тестүүдийг мөрдөх.

Хэсгүүд	X	z, o	Өртөг	Тоо
`10 0 010 0 0`	5	3, 0	13	c(2) · c(3) · Bell(3) · Bell(0) = 5
`0 0 01 1 1`	2	2, 2	8	c(2) · Bell(2) · Bell(2) = 4
`01010101`	8	0, 0	16	c(8) · Bell(0) · Bell(0) = 4

5.9 Код

Код 5.1. E.cpp, 1–21-р мөр: Беллийн гурвалжин

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    const long long MOD = 998244353;
    const int MAXN = 5000;

    vector<long long> bell(MAXN + 1);
    vector<long long> row = {1};
    bell[0] = 1;
    for (int i = 1; i <= MAXN; i++) {
        vector<long long> next;
        next.reserve(i + 1);
        next.push_back(row.back());
        for (long long x : row) next.push_back((next.back() + x) % MOD);
        bell[i] = next[0];
        row = move(next);
    }
```

`row` нь гурвалжны одоогийн мөр. 17-р мөр нь өмнөх мөрийн сүүлчийн тоогоор эхэлж, 18-р мөр нь нэмэх дүрмийг хэрэгжүүлнэ. 16-р мөрийн `reserve` нь зөвхөн хурдны төлөө.

Код 5.2. E.cpp, 23–28-р мөр: $c(\ell)$ хүснэгт

```cpp
vector<long long> ways(MAXN + 1, 0);
ways[0] = 1;
for (int len = 2; len <= MAXN; len++) {
    ways[len] = ways[len - 2];
    if (len >= 3) ways[len] = (ways[len] + ways[len - 3]) % MOD;
}
```

Код 5.3. `E.cpp`, 30–59-р мөр: тест бүрийг $O(n)$ хугацаанд бодох

```
    int t;
    cin >> t;
    while (t--) {
        int n;
        string s;
        cin >> n >> s;

        int single[2] = {0, 0};
        int covered = 0;
        long long cnt = 1;

        for (int i = 0; i < n;) {
            int j = i;
            while (j + 1 < n && s[j + 1] != s[j]) j++;
            int len = j - i + 1;

            if (len == 1) {
                single[s[i] - '0']++;
            } else {
                covered += len;
                cnt = cnt * ways[len] % MOD;
            }
            i = j + 1;
        }

        cnt = cnt * bell[single[0]] % MOD * bell[single[1]] % MOD;
        cout << n + covered << ' ' << cnt << '\n';
    }
    return 0;
}
```

- **41–44-р мөр.** `i`-ээс эхэлсэн ээлжилсэн хэсгийн төгсгөл `j`-г олно. Дараагийн хэсэг нь `j + 1`-ээс эхлэх тул гадна давталт нийт $O(n)$ алхам хийнэ.
- **46–51-р мөр.** Теорем 5.4 ба Теорем 5.5-ын томьёог шууд хэрэгжүүлнэ. `s[i] - '0'` нь тэмдэгтийг 0 эсвэл 1 индекс болгоно.
- **55-р мөр.** Хоёр үржүүлэх бүрийн дараа модуль авдаг тул `long long` халихгүй.

5.10 Хугацаа ба анхаарах зүйл

Хүснэгтүүдийг нэг удаа $O(N^2)$ хугацаанд ($N = 5000$, $1.25 \cdot 10^7$ нэмэх үйлдэл) бодно. Тест бүр $O(n)$. 500 удаа $n = 5000$ тест бүхий оролтод 0.2 секунд хүрэхгүй хугацаа зарцуулсан (Хавсралт А).

> **Өмнөх хувилбараас авах сургамж**
>
> Энэ бодлогын анхны шийдэл нь (байрлал, холимог олонлогт орсон 0-ийн тоо) төлөвтэй динамик программчлал байсан. Хариу нь зөв, жижиг тестийг бүгдийг давдаг ч нэг тестэд $O(n^2)$ тул 500 том тестэд 20 секунд орчим зарцуулж, 2 секундийн хязгаарыг давдаг байв. Теорем 5.4-ийн ажиглалтаар холимог олонлогт орохгүй байрлалууд тогтмол болж тэр төлөв огт хэрэггүй болсон. Хязгаарт нийлбэрийн заалт байхгүй бол үүнийг заавал анхаарах хэрэгтэй.

Анхаарах

Нэгэн төрлийн хэсгийг мартаж болохгүй. Ганц хэсгийн байрлалуудыг зөвхөн тус тусад нь биш, ижил тэмдэгттэйг нь дурын байдлаар нэгтгэж болно. Беллийн тоо эндээс гарч ирдэг.

0 ба 1-ийг тусад нь. Bell($z + o$) биш, Bell(z) · Bell(o). Өөр тэмдэгттэй хоёр ганц байрлал нэг олонлогт орвол Лемм 5.1-ийг зөрчинө.

Дүгнэлт

Оновчтой шийдлүүдийн тоог асуусан бодлогод эхлээд оновчтой шийдлийн **бүтцийг** бүрэн тодорхойлоорой. Бүтэц тодорхой болмогц тоолол нь бие даасан сонголтуудын үржвэр болж, төлөв ихтэй DP шаардлагагүй болно.

Дасгал

5.1. s = 0110 үед хамгийн их өртөг ба хуваалтын тоог ол.

5.2. s = 00000 үед хариу юу вэ?

БҮЛЭГ 6

F. Minimize the Graph

Графыг багасгах

Сэдэв цикл-гүй чиглэлт граф, транзитив багасгалт **Файл** `F.cpp`

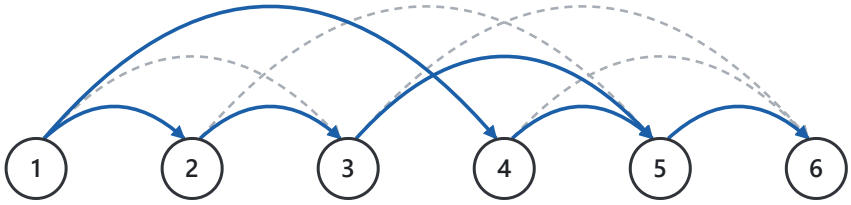
Хэрэгсэл нэгтгэх-хайх бүтэц (DSU), Фенвикийн мод

Ирмэгийн тоо n^2-д хүрч болох ч ирмэгүүд интервал хэлбэрээр өгөгддөг. Хамгийн цөөн ирмэгтэй граф нь онолын хувьд сайн мэдэгдэх объект бөгөөд түүнийг интервалын хэлбэр дээр энгийн томьёогоор тоолж болно.

6.1 Бодлого

n оройтой чиглэлт граф өгөгдөнө. $i = 2, \ldots, n$ бүрт $1 \le l_i \le r_i < i$ байх хоёр тоо өгөгдөх ба $[l_i, r_i]$ доторх j бүрээс i рүү ирмэг татагдана. Өөр ирмэг байхгүй.

Ижил оройнууд дээр хоёр дахь граф байгуулна. Ялгаатай дурын x, y-ийн хувьд x-ээс y рүү зам байх эсэх нь хоёр графт ижил байх ёстой. Хоёр дахь графын **хамгийн цөөн ирмэгийн тоог** ол. **Хязгаар:** $n \le 3 \cdot 10^5$, n-ийн нийлбэр $3 \cdot 10^5$-аас хэтрэхгүй.



Зураг 6.1. $n = 6$; $[l_i, r_i]$ нь [1,1], [1,2], [1,1], [2,4], [3,5]. Цэнхэр ирмэгүүд нь хамгийн цөөн ирмэгтэй графыг бүрдүүлнэ. Тасархай ирмэгийг хасаж болно, жишээ нь 1→3 ирмэгийг 1→2→3 зам орлоно. Хариу 6.

6.2 Хамгийн цөөн ирмэгтэй граф

Бүх ирмэг бага дугаартай оройноос их дугаартай руу чиглэсэн тул граф нь цикл-гүй. $u \rightsquigarrow v$ нь u-ээс v рүү (дор хаяж нэг ирмэгтэй) зам байгааг илэрхийлнэ.

Тодорхойлолт 6.1. $u \rightsquigarrow v$ бөгөөд $u \rightsquigarrow w \rightsquigarrow v$ байх завсрын w орой байхгүй бол (u, v)-г **шууд хос** гэнэ.

Теорем 6.2. Хамгийн цөөн ирмэгийн тоо нь шууд хосуудын тоотой тэнцүү. Шууд хос бүр анхны графын ирмэг юм.

Баталгаа. H нь шаардлагыг хангах дурын граф, (u, v) шууд хос гэе. H дахь u-ээс v рүү энгийн замд завсрын орой w байсан бол $u \rightsquigarrow w \rightsquigarrow v$ нь H-д, тиймээс анхны графт ч биелнэ. Энэ нь зөрчилтэй. Иймд зам нь ганц ирмэг буюу H бүх шууд хосыг агуулна. Яг ижил үндэслэлээр анхны графт ч шууд хос нь ирмэг байна.

Эсрэгээр, зөвхөн шууд хосуудаас тогтох граф хангалттай. $u \rightsquigarrow v$ бол анхны граф дахь u-ээс v хүртэлх хамгийн **урт** замыг авъя. Түүний (a, b) ирмэг бүр шууд хос, эс бөгөөс $a \rightsquigarrow w \rightsquigarrow b$-ээр замыг уртасгах боломжтой. ■

Энэ графыг **транзитив багасгалт** гэдэг. Тиймээс бодлого нь: **анхны графын ирмэгүүдээс хэд нь шууд хос вэ?**

6.3 Аль ирмэгийг үлдээх вэ

Лемм 6.3. $j \to i$ ирмэг ($j \in [l_i, r_i]$) шууд хос биш байх нь j-ээс $(j, r_i]$ мужийн ямар нэг оройд хүрч болохтой ижил.

Баталгаа. j-ээс $x \in (j, r_i]$ оройд хүрч чадвал $x \geq l_i$ учир $x \to i$ ирмэг бий. Тэгэхээр $j \rightsquigarrow x \rightsquigarrow i$ байх тул шууд хос биш.

Эсрэгээр, $j \rightsquigarrow w \rightsquigarrow i$ гэе. w-ээс i хүрэх замын сүүлчийн ирмэг $x \to i$ бол $x \in [l_i, r_i]$ ба $j \rightsquigarrow x$. Зам үргэлж их дугаар руу явдаг тул $x > j$, өөрөөр хэлбэл $x \in (j, r_i]$. ■

Одоо $(j, r_i]$-д хүрч болох эсэхийг мэдэхийн тулд j-ээс хүрч болох **хамгийн бага** дугаартай оройг мэдэхэд л хангалттай. Энэ орой маш энгийнээр тодорхойлогддог. $u \to v$ ирмэг байвал v-г u-ийн **гарах хөрш** гэж нэрлэе.

Лемм 6.4. j-ээс хүрч болох хамгийн бага орой нь түүний хамгийн бага гарах хөрш буюу $\mathrm{nxt}(j) = \min\{x : l_x \leq j \leq r_x\}$.

Баталгаа. j-ээс y хүрэх дурын замын эхний ирмэг $j \to c$ бол зам өсөх чиглэлтэй тул $c \leq y$. c нь гарах хөрш учир $\mathrm{nxt}(j) \leq c \leq y$. Мөн $\mathrm{nxt}(j)$ өөрөө хүрч болох орой. ■

Теорем 6.5. Хариу нь $\Sigma_{i=2}^{n} |\{j \in [l_i, r_i] : \mathrm{nxt}(j) > r_i\}|$. Гарах хөршгүй j-д $\mathrm{nxt}(j) = \infty$ гэж үзнэ.

Баталгаа. Теорем 6.2, Лемм 6.3 ба Лемм 6.4-ийг нэгтгэхэд шууд гарна. ■

Хүснэгт 6.1. Зураг 6.1-ийн жишээг мөрдөх. Нийт $1 + 1 + 1 + 2 + 1 = 6$.

i	$[l_i, r_i]$	r_i	$\mathrm{nxt}(j) > r_i$ эсэх	Үлдэх
2	[1, 1]	1	nxt(1) = 2 ✓	1
3	[1, 2]	2	nxt(1) = 2 ✗, nxt(2) = 3 ✓	1
4	[1, 1]	1	nxt(1) = 2 ✓	1
5	[2, 4]	4	nxt(2) = 3 ✗, nxt(3) = 5 ✓, nxt(4) = 5 ✓	2
6	[3, 5]	5	nxt(3) = 5 ✗, nxt(4) = 5 ✗, nxt(5) = 6 ✓	1

6.4 Алгоритм ба хэрэгслүүд

1. $x = 2, \ldots, n$ дарааллаар $[l_x, r_x]$ доторх оноогоогүй j бүрт $\mathrm{nxt}(j) = x$ гэж оноох.
2. j-үүдийг $\mathrm{nxt}(j)$-ээр, i-үүдийг r_i-ээр бүлэглэх.

3. $R = n, n-1, \ldots, 1$ дарааллаар явж, nxt(j) > R болсон j-г модонд нэмээд $r_i = R$ бүрт муж доторх тоог хариунд нэмэх.

Томьёог хэрэгжүүлэхэд хоёр хэрэгсэл хэрэгтэй: nxt-ийг хурдан бодох, мөн муж доторх тоолол.

Хэрэгсэл · nxt-ийг нэгтгэх-хайх бүтцээр бодох

$x = 2, 3, \ldots, n$ дарааллаар явахад $[l_x, r_x]$ доторх **оноогоогүй** j бүрт nxt(j) = x болно. Дараалал нь өсөх тул эхэлж оноогдсон x хамгийн бага нь. j бүр нэг л удаа оноогддог ч оноогдсон оройнуудыг давж гарах хэрэгтэй.

Үүнд `find(j)` функц ашиглана: энэ нь j-ээс эхлээд оноогоогүй хамгийн бага индексийг буцаана. j оноогдоход `dsu[j] = j + 1` гэж заагч тавьдаг тул дараагийн хайлт шууд үсэрч гарна. Замыг богиносгох тул нийт хугацаа бараг шугаман.

Хүснэгт 6.2. Зураг 6.1-ийн жишээ дээр nxt-ийг бодох. Үр дүн: nxt(1) = 2, nxt(2) = 3, nxt(3) = 5, nxt(4) = 5, nxt(5) = 6, nxt(6) = ∞.

x	$[l_x, r_x]$	Оноогдсон j	Тайлбар
2	[1, 1]	1	
3	[1, 2]	2	
4	[1, 1]	—	бүгд оноогдсон
5	[2, 4]	3, 4	
6	[3, 5]	5	

Хэрэгсэл · Фенвикийн мод ба оффлайн тоолол

Фенвикийн мод нь `add(p, d)` (нэг байрлалд нэмэх) ба `pref(p)` (эхний p байрлалын нийлбэр) үйлдлийг $O(\log n)$ хугацаанд хийдэг массив бүтэц. `p & -p` нь p-ийн хамгийн бага 1 битийг гаргаж авах ба `add` энэ утгаар дээш, `pref` доош алхаж log n-ээс олонгүй нүдэд хандана.

i бүрт $[l_i, r_i]$ доторх nxt(j) > r_i байх j-г тоолох хэрэгтэй. Асуултуудыг r_i-гаар нь бүлэглэж, $R = n, n-1, \ldots, 1$ дарааллаар явъя. R буурах тусам “nxt(j) > R” нөхцөлийг хангах j зөвхөн нэмэгдэнэ. Тиймээс nxt(j) = R + 1 болсон j-г модонд нэмээд, муж доторх нийлбэрийг `pref(r) - pref(l - 1)` -ээр авна. Асуултын ирсэн дарааллаар биш, бидэнд тохиромжтой дарааллаар боловсруулах энэ аргыг **оффлайн** гэдэг.

6.5 Код

Код 6.1. F.cpp, 1–14-р мөр: ороолт

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;
    while (t--) {
        int n;
        cin >> n;
        vector<int> l(n + 1, 0), r(n + 1, 0);
        for (int i = 2; i <= n; i++) cin >> l[i] >> r[i];
```

Код 6.2. F.cpp, 16–30-р мөр: nxt-ийг бодох

```cpp
vector<int> nxt(n + 2, n + 1), dsu(n + 2);
for (int j = 0; j <= n + 1; j++) dsu[j] = j;
auto find = [&](int x) {
    while (dsu[x] != x) {
        dsu[x] = dsu[dsu[x]];
        x = dsu[x];
    }
    return x;
};
for (int x = 2; x <= n; x++) {
    for (int j = find(l[x]); j <= r[x]; j = find(j)) {
        nxt[j] = x;
        dsu[j] = j + 1;
    }
}
```

- **16-р мөр.** nxt-ийн анхны утга $n + 1$ нь ∞-ийн үүрэг гүйцэтгэнэ.
- **18–24-р мөр.** `find`-ийн `dsu[x] = dsu[dsu[x]]` мөр нь замыг хоёр дахин богиносгож, цаашдын хайлтыг хурдасгана.
- **26-р мөр.** `find(j)` нь оноогдсон оройнуудыг алгасна. `j = find(j)` нь `dsu[j] = j + 1` тавигдсан тул дараагийн сул индекс рүү шилжинэ.

Код 6.3. F.cpp, 32–44-р мөр: бүлэглэх ба Фенвикийн мод

```cpp
vector<vector<int>> byNxt(n + 2), byR(n + 1);
for (int j = 1; j <= n; j++) byNxt[nxt[j]].push_back(j);
for (int i = 2; i <= n; i++) byR[r[i]].push_back(i);

vector<long long> bit(n + 1, 0);
auto add = [&](int p, long long d) {
    for (; p <= n; p += p & -p) bit[p] += d;
};
auto pref = [&](int p) {
    long long res = 0;
    for (; p > 0; p -= p & -p) res += bit[p];
    return res;
};
```

Код 6.4. F.cpp, 46–56-р мөр: оффлайн тоолол

```
46          for (int j : byNxt[n + 1]) add(j, 1);
47          long long total = 0;
48          for (int R = n; R >= 1; R--) {
49              if (R + 1 <= n)
50                  for (int j : byNxt[R + 1]) add(j, 1);
51              for (int i : byR[R]) total += pref(r[i]) - pref(l[i] - 1);
52          }
53          cout << total << '\n';
54      }
55      return 0;
56  }
```

- **46-р мөр.** Гарах хөршгүй оройнуудыг (nxt = n + 1) эхэнд нь нэмнэ. Эдгээр нь бүх R-д нөхцөлийг хангана.
- **49–50-р мөр.** R нэгээр буурахад nxt(j) = R + 1 оройнууд нэмэгдэнэ.
- **51-р мөр.** Теорем 6.5-ын нийлбэрийг бодно. Мужаас гадуурх $j > r_i$ оройнууд модонд байж болох ч `pref` зөвхөн мужийн доторхыг тоолно.

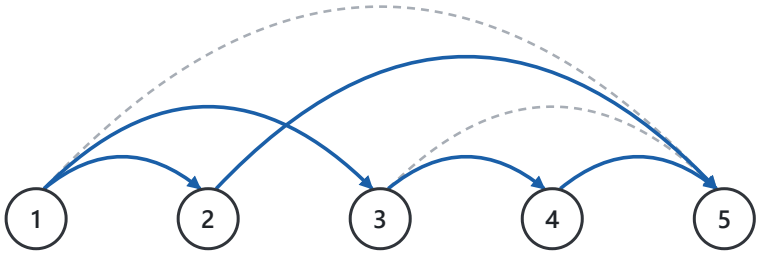
6.6 Хугацаа ба анхаарах зүйл

nxt-ийг бодох нь бараг $O(n)$, Фенвикийн модны $O(n)$ үйлдэл тус бүр $O(\log n)$. Нийт $O(n \log n)$ хугацаа, $O(n)$ санах ой.

> **Анхаарах**
>
> **Хариу том байж болно.** Хоёр бүлэг орой бүрэн холбогдсон үед хариу ойролцоогоор $n^2 / 4 \approx 2.25 \cdot 10^{10}$ тул `long long` заавал хэрэгтэй.
>
> **Ирмэгүүдийг шууд үүсгэж болохгүй.** Нийт ирмэгийн тоо n^2-д хүрч болох тул интервалын томьёогоор тоолох шаардлагатай.

6.7 Нэгэн алдааны түүх

Энэ бодлогын анхны шийдэл бүх жишээг зөв бодож байсан ч буруу байв. Шалтгаан нь нэг батлагдаагүй таамаг: “i-д хүрч чадах оройнууд тасралтгүй интервал үүсгэнэ” гэж үзсэн. Иймд nxt(j)-г олохдоо $[l_x, r_x]$-ийн оронд x-д хүрч чадах бүх оройн мужийг ашигласан.



Зураг 6.2. Алдааг илрүүлсэн хамгийн жижиг оролт: [1,1], [1,1], [3,3], [1,4]. Зөв хариу 5, алдаатай код 4 гаргасан.

Зураг 6.2-т 4-р оройд зөвхөн 1 ба 3-р оройгоос хүрч болно. 2-р орой 4-т хүрэхгүй! Гэвч буруу таамгаар [1, 3] интервал гэж үзсэн тул nxt(2) = 4 болж, хэрэгтэй 2→5 ирмэгийг хасжээ. Зөв

нь nxt(2) = 5.

Энэ алдааг жишээ тестүүд барьж чадаагүй. Жижиг оролтуудыг бүрэн хайлттай харьцуулах стресс тест л илрүүлсэн. $n \le 4$-т нэг ч алдаа гараагүй, харин $n = 5$-ын 180 оролтын 2-т нь хариу зөрсөн. Хамгийн сайхан нь засвар шийдлийг бүр хялбар болгосон: Лемм 6.4-ийн ёсоор хол явах шаардлага огт байгаагүй.

> **Дүгнэлт**
> Цөөн ирмэгтэй графын асуудал нь зам ба завсрын оройн тухай цэвэр онолд буудаг. Интервал хэлбэрийн оролтод оройг нэг бүрчлэн биш, “хамгийн бага гарах хөрш” шиг нэг тоогоор төлөөлүүлж чадах эсэхээ шалгаарай. Мөн таамаг бүрээ баталж эсвэл стресс тестээр шалгаж байх хэрэгтэй.

> **Дасгал**
> **6.1.** $n = 4$; [1, 1], [1, 2], [2, 3] үед nxt-ийг бодож хариуг ол.
> **6.2.** $n = 5$; [1, 1], [1, 1], [2, 3], [2, 3] үед аль ч ирмэгийг хасаж болох уу?

БҮЛЭГ 7

G. Order Statistics on the Graph

Граф дээрх эрэмбийн статистик

Сэдэв босгоор задлах, функцийн давталт **Файл** `G.cpp` **Хэрэгсэл** хоёрын зэргээр үсрэх (binary lifting)

t нь 10^{18} хүртэл, утгууд нь дурын, асуулга нь 10^5 байна. Гэвч $n \le 20$ гэдэг жижиг хязгаар сэжүүр өгнө: утгуудыг 0, 1 болгож задалбал боломжит төлөвийн тоо хамгийн ихдээ 2^{20} болно.

7.1 Бодлого

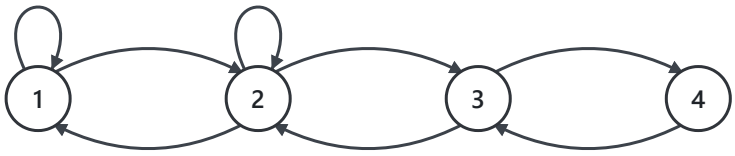
n оройтой чиглэлт граф өгөгдөнө. Орой бүрт дор хаяж нэг орох ирмэг байх ба өөртөө чиглэсэн ирмэг байж болно. v орой бүрт $1 \le k_v \le d_v$ бүхэл тоо өгөгдөнө, энд d_v нь орох ирмэгийн тоо.

Орой бүр дээр нэг тоо бичигдсэн. Секунд бүрт бүх орой **зэрэг** шинэчлэгдэнэ: v-ийн шинэ утга нь орох хөршүүдийн (өөр дээрээ ирмэгтэй бол өөрийгөө оролцуулан) утгуудын k_v-р хамгийн бага утга буюу эрэмбийн статистик.

q асуулга бүрт t ба анхны утгууд $a_1, \dots, a_n$ өгөгдөнө. t секундийн дараах утгуудыг хэвлэ. **Хязгаар:** $n \le 20$, $q \le 10^5$, $t \le 10^{18}$, $a_i \le 10^9$.

7.2 Жишээгээр ойлгох

Гурав дахь жишээг авч үзье. Граф нь Зураг 7.1-д, орой тус бүрийн үүрэг нь Хүснэгт 7.1-д байна.



Зураг 7.1. Жишээ граф: 1–2, 2–3, 3–4 оройнууд хоёр чиглэлд холбоотой, 1 ба 2-р оройд өөртөө чиглэсэн ирмэг бий.

Хүснэгт 7.1. Оройнуудын үүрэг. c_v-г дараагийн хэсэгт тайлбарлана.

v	Орох хөршүүд	d_v	k_v	$c_v = d_v - k_v + 1$	Утга
1	1, 2	2	1	2	хамгийн бага
2	1, 2, 3	3	2	2	медиан (дунд утга)
3	2, 4	2	2	1	хамгийн их
4	3	1	1	1	3-р оройн утгыг хуулах

$a = [8, 3, 5, 1]$ үед нэг секундийн дараа $v_1 = \min(8, 3) = 3$, v_2 нь 8, 3, 5-ын медиан 5, $v_3 = \max(3, 1) = 3$, $v_4 = 5$ буюу $[3, 5, 3, 5]$. Хоёр секундийн дараа $[3, 3, 5, 3]$, гурван секундийн дараа $[3, 3, 3, 5]$.

Шууд загварчлах нь t-ийн улмаас боломжгүй. Утгын вектор давтагддаг ч давталтын урт болон асуулгын тоо их.

7.3 Босгоор задлах

Тогтмол x утгын хувьд “утга нь x-ээс багагүй юу?” гэсэн асуултыг орой бүрт тавья. Хариу нь 1 эсвэл 0 бит. Гол нээлт нь эдгээр битүүд утгуудаас үл хамааран хувьсдаг явдал юм.

Лемм 7.1. $c_v = d_v - k_v + 1$ гэе. v-ийн шинэ утга x-ээс багагүй байх нь x-ээс багагүй утгатай орох хөршийн тоо c_v-ээс багагүй байхтай ижил.

Баталгаа. Орох хөршүүдийн утгыг өсөх дарааллаар эрэмбэлэе. k_v-р утга x-ээс багагүй бол k_v, $k_v + 1$, …, d_v байрлалын $d_v - k_v + 1$ утга бүгд x-ээс багагүй. Эсрэгээр, ядаж ийм олон утга x-ээс багагүй бол тэдгээр нь эрэмбэд хамгийн сүүлд байх тул k_v-р байрлал тэдний дотор орно. ■

Энэ нь чухал үр дүнд хүргэнэ. Битийн вектор s-ийг нэг бүхэл тоо (маск) гэж харвал дараагийн төлөв нь зөвхөн s ба графаас хамаарна:

$$F(s)_v = 1 \iff \text{popcount}(s \,\&\, in_v) \geq c_v$$

Энд in_v нь v-ийн орох хөршүүдийн маск. F функц утгууд ба x-ээс **хамаарахгүй**, тиймээс бүх асуулга ба бүх босгонд нэг л хүснэгт хэрэглэгдэнэ. $n \leq 20$ тул $2^n \leq 1048576$ төлөвт F-ийг урьдчилан бодож болно.

Хүснэгт 7.2. $a = [8, 3, 5, 1]$-ийн босго бүрийн бит, нэг алхмын дараа.

Босго x	s: $v_1\,v_2\,v_3\,v_4$	$F(s)$: $v_1\,v_2\,v_3\,v_4$
1	1111	1111
3	1110	1111
5	1010	0101
8	1000	0000

Сүүлийн баганыг босоогоор нь уншвал утгууд сэргээгдэнэ. v_2-ын бит $x = 1, 3, 5$-д 1, $x = 8$-д 0 тул шинэ утга нь 5. Энэ нь үнэхээр $[3, 5, 3, 5]$-тай таарч байна.

7.4 Утгыг битүүдээс сэргээх

Анхны утгуудыг эрэмбэлж давхардлыг хассан жагсаалтыг xs гэе. Эрэмбийн статистик нь үргэлж аль нэг утгыг сонгодог тул t секундийн дараа утга бүр xs-д байна.

Лемм 7.2. xs-ийн босго бүрээр F^t-г бодоод v-ийн бит 1 байсан босгын тоог cnt_v гэвэл t секундийн дараах v-ийн утга $\text{xs}[\text{cnt}_v]$ байна (xs-ийг 1-ээс дугаарлая).

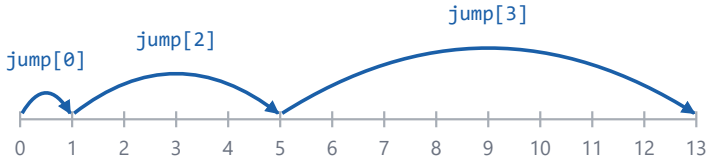
Баталгаа. Лемм 7.1-ийг t удаа хэрэглэвэл босго x-ийн F^t-ийн v-р бит нь “t секундийн дараах v-ийн утга $\geq x$” гэсэн үг. Эцсийн утга xs-д байгаа тул түүнээс багагүй xs-ийн утгын тоо нь яг түүний эрэмбэ болно. ■

7.5 Хоёрын зэргээр үсрэх

Хэрэгсэл · хоёрын зэргээр үсрэх (binary lifting)

F^t-г t удаа давтаж бодох боломжгүй. Оронд нь jump[b][s] = $F^{2^b}(s)$ хүснэгт байгуулна. jump[0] нь F өөрөө, цааш нь $F^{2^b} = F^{2^{b-1}} \circ F^{2^{b-1}}$ буюу `jump[b][s] = jump[b-1][jump[b-1][s]]`.

t-г хоёртын системд задлаад 1 бит бүрт харгалзах үсрэлтийг хийнэ. $10^{18} < 2^{60}$ тул 60 түвшин хангалттай.



Зураг 7.2. t = 13 = 1 + 4 + 8 үед гурван үсрэлт хангалттай.

7.6 Алгоритм ба жишээ

1. Орох хөршүүдийн маск in_v ба c_v-г бодно.
2. Бүх s-д jump[0][s] = $F(s)$, дараа нь бүх b < 60-д jump[b]-г бодно.
3. Асуулга бүрт утгуудыг эрэмбэлж xs-г олно.
4. xs-ийн босго x бүрт анхны битийн маскийг үүсгэж, t-ийн 1 бит бүрээр үсэрнэ.
5. 1 болсон бит бүрт cnt_v-г нэмнэ. v-ийн хариу нь xs[cnt_v].

Хүснэгт 7.3. a = [8, 3, 5, 1], t = 2.
xs = [1, 3, 5, 8]. Хариу [3, 3, 5, 3].

Босго x	s	$F(s)$	$F^2(s)$
1	1111	1111	1111
3	1110	1111	1111
5	1010	0101	0010
8	1000	0000	0000
cnt_v			2 2 3 2
утга			3 3 5 3

7.7 Код

Код 7.1. G.cpp, 1–24-р мөр: граф ба c_v

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m, q;
    cin >> n >> m >> q;
    vector<int> k(n);
    for (auto &x : k) cin >> x;

    vector<int> inmask(n, 0);
    for (int e = 0; e < m; e++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        inmask[v] |= 1 << u;
    }

    vector<int> need(n);
    for (int v = 0; v < n; v++)
        need[v] = __builtin_popcount(inmask[v]) - k[v] + 1;
```

Кодод оройг 0-ээс дугаарладаг тул маскийн v-р бит нь v + 1-р оройг заана. `need[v]` нь c_v. `__builtin_popcount` нь тооны 1 битийг тоолдог GCC-ийн функц.

Код 7.2. G.cpp, 26–36-р мөр: F ба үсрэлтийн хүснэгт

```cpp
const int S = 1 << n;
const int LOG = 60;
vector<vector<int>> jump(LOG, vector<int>(S));
for (int s = 0; s < S; s++) {
    int ns = 0;
    for (int v = 0; v < n; v++)
        if (__builtin_popcount(s & inmask[v]) >= need[v]) ns |= 1 << v;
    jump[0][s] = ns;
}
for (int b = 1; b < LOG; b++)
    for (int s = 0; s < S; s++) jump[b][s] = jump[b - 1][jump[b - 1][s]];
```

- **29–34-р мөр.** Төлөв бүрт Лемм 7.1-ийн нөхцлийг шалгаж $F(s)$-г бодно.
- **35–36-р мөр.** Өмнөх түвшнийг хоёр удаа хэрэглэж дараагийн түвшнийг гаргана.

Код 7.3. `G.cpp`, 38–66-р мөр: асуулгуудад хариулах

```
    string out;
    while (q--) {
        long long T;
        cin >> T;
        vector<long long> a(n);
        for (auto &x : a) cin >> x;

        vector<long long> xs = a;
        sort(xs.begin(), xs.end());
        xs.erase(unique(xs.begin(), xs.end()), xs.end());

        vector<int> cnt(n, 0);
        for (long long x : xs) {
            int s = 0;
            for (int v = 0; v < n; v++)
                if (a[v] >= x) s |= 1 << v;
            for (int b = 0; b < LOG; b++)
                if ((T >> b) & 1) s = jump[b][s];
            for (int v = 0; v < n; v++)
                if ((s >> v) & 1) cnt[v]++;
        }
        for (int v = 0; v < n; v++) {
            out += to_string(xs[cnt[v] - 1]);
            out += (v + 1 < n) ? ' ' : '\n';
        }
    }
    cout << out;
    return 0;
}
```

- **45–47-р мөр.** Утгуудыг эрэмбэлж давхардлыг арилгана. Эдгээр нь шалгах босгонууд.
- **51–53-р мөр.** Босго x-ийн анхны маск.
- **54–55-р мөр.** t-ийн 1 бит бүрээр үсэрнэ. Функцийн зэргүүд солигдох чанартай тул дараалал хамаагүй.
- **60-р мөр.** Массив 0-ээс эхэлдэг тул `xs[cnt[v] - 1]`. Хамгийн бага босгод бүх бит 1 тул `cnt[v] ≥ 1`.

7.8 Хугацаа ба анхаарах зүйл

F-г бодоход $O(2^n \cdot n)$, хүснэгтэд $O(2^n \cdot 60)$. Асуулга бүрт хамгийн ихдээ n босго, тус бүр 60 хүртэл үсрэлт. Хамгийн муу үед $10^5 \cdot 20 \cdot 60 = 1.2 \cdot 10^8$ хандалт. Санах ой $60 \cdot 2^{20} \cdot 4$ байт буюу 240 MB. Хэмжилтээр хугацаа 2 секунд орчим, санах ой 280 MB орчим (хязгаар 7 секунд, 512 MB), Хавсралт А-г үзнэ үү.

Анхаарах

c_v нь k_v биш. $k_v = 1$ (хамгийн бага) үед бүх хөрш x-ээс багагүй байх ёстой тул $c_v = d_v$. $k_v = d_v$ (хамгийн их) үед нэг нь хангалттай тул $c_v = 1$.

Ижил утгууд. Давхардлыг заавал арилгана, эс бөгөөс `cnt` буруу болно. Дээрх лемм нь "$\geq x$" хэлбэрээр тэнцүү утгуудыг аль хэдийн зөв шийддэг.

$t = 0$. Нэг ч үсрэлт хийхгүй тул анхны утгууд буцна.

`long long`. t нь `int`-д багтахгүй. `T >> b` зөв ажиллахын тулд `T` нь 64 бит байх ёстой.

Дүгнэлт

Эрэмбэ, медиан, хамгийн их, хамгийн бага гэх мэт үйлдэлд зөвхөн утгуудын харьцуулалт чухал. Иймээс босгоор 0 ба 1 болгож задлах арга маш хүчтэй. Нэгэнт төлөвийн тоо хязгаартай болмогц функцийн давталтыг хоёрын зэргээр үсрэх аргаар асар том t-д ч хурдан тооцож болно.

Дасгал

7.1. $d_v = 5$, $k_v = 2$ бол c_v хэд вэ? Утгаар нь тайлбарла.

7.2. Жишээ граф дээр $a = [8, 3, 5, 1]$, $t = 3$ үеийн хариуг босгоор бодож ол.

Хавсралт A. Шалгалтын тайлан

Энэ хавсралтад шийдлүүдийг хэрхэн шалгасныг нэгтгэв. Шалгалт бүрийг номын эцсийн кодоор ажиллуулсан. Анхаарах нэг зүйл: **шийдлүүдийг Codeforces-ийн серверт илгээж шалгаагүй.** Доорх бүх дүн энэ компьютер дээр хийсэн туршилтын дүн юм.

Бүрэн хайлттай харьцуулах

Жижиг оролт дээр бодлогын тодорхойлолтыг шууд хэрэгжүүлсэн удаан програм бичиж, шийдлийн гаралттай харьцуулсан. Бүх мөрөнд зөрүү **0** гарсан. Мөн бодлого бүрийн жишээ оролт зөв гарсан.

Хүснэгт A.1. Бүрэн хайлттай харьцуулсан шалгалтууд.

Бодлого	Юутай харьцуулсан	Хэмжээ
A	Тэг нийлбэртэй олонлогийг DP-ээр хайх	$-20 \le l \le r \le 20$, бүх 861 хос
B	Мөрийг бодитоор үүсгэж, үеийг тодорхойлолтоор шалгах	400 санамсаргүй массив
C	$m \ge 2k$ нөхцөлийг шууд тооцох (хоёр хувилбар)	$n = 1..2000$ ба том n, нийт 2023
D	Үйлдлүүдийг яг дарааллаар нь загварчлах, k бүрээр (хоёр хувилбар)	$n \le 7$ бүх 18247 мод; 3000 санамсаргүй мод, $n \le 13$
E	Бүх зөв хуваалтыг шууд тоолох	$n \le 8$ бүх мөр ба $n = 9$ сонголт, нийт 550
E	Өмнөх $O(n^2)$ DP хувилбар	400 санамсаргүй мөр, $n \le 400$
F	Транзитив багасгалтыг тодорхойлолтоор тооцох	$n \le 7$ бүх 59602 оролт; 4000 санамсаргүй, $n \le 16$
F	Боломжит бүх ирмэгийн олонлогоос хамгийн цөөнийг хайх	$n \le 5$ бүх 202 оролт
G	Утгуудыг шууд загварчилж, давталтыг илрүүлэн t-г богиносгох	9000 асуулга, $n \le 6$, $t \le 10^{18}$

Хамгийн их оролт дээрх хугацаа

Дараах хүснэгтэд нөхцөлийн хязгаарт тулсан оролт дээр хэмжсэн хугацаа, санах ойг үзүүлэв (`g++ -O2`, энэ компьютер). Гаралтыг файлд бичээгүй тул бодит хугацаа арай өөр байж болно. Санах ойн хязгаар бүх бодлогод 512 MB.

Хүснэгт A.2. Хамгийн их оролтын хэмжилт.

Бодлого	Ролт	Хугацаа	Хязгаар	Санах ой
A	$t = 10^4$, санамсаргүй	0.08 с	2 с	5 MB
B	$n = 5000$, санамсаргүй	0.12 с	2 с	5 MB
B	$n = 5000$, бүх утга ижил	0.15 с	2 с	5 MB
C	$n = 2 \cdot 10^5$	0.93 с	2 с	6 MB
C	$t = 10^4$, $n = 20$	0.75 с	2 с	5 MB
D	$n = 299999$, зам хэлбэрийн мод	0.37 с	2 с	24 MB
D	$n = 299999$, од хэлбэрийн мод	0.12 с	2 с	25 MB
D	$n = 299999$, санамсаргүй мод	0.27 с	2 с	26 MB
D	$t = 10^4$, $n = 29$	0.15 с	2 с	5 MB
E	$t = 500$, $n = 5000$, ээлжилсэн мөр	0.15 с	2 с	5 MB
E	$t = 500$, $n = 5000$, санамсаргүй мөр	0.12 с	2 с	5 MB
F	$n = 3 \cdot 10^5$, $l_i = 1$, $r_i = i - 1$	0.18 с	3 с	35 MB
F	$n = 3 \cdot 10^5$, хоёр бүлэг бүрэн холбоотой	0.10 с	3 с	28 MB
F	$n = 3 \cdot 10^5$, санамсаргүй	0.25 с	3 с	30 MB
G	$n = 20$, $m = 400$, $q = 10^5$	1.93 с	7 с	275 MB
G	$n = 20$, $m \approx 40$, $q = 10^5$	2.00 с	7 с	277 MB

> **Энэ шалгалтаас гарсан хоёр сургамж**
>
> E бодлогын өмнөх хувилбар бүх жижиг тестийг зөв давж байсан ч дээд ролт дээр 20 секунд орчим зарцуулсан тул зөвхөн хариуг биш, хурдыг ч шалгах ёстойг харуулсан. F бодлогын анхны хувилбар бүх жишээг давсан ч бүрэн хайлт $n = 5$ дээр алдааг нь илрүүлсэн (6.7-р хэсэг).

Хавсралт Б. Нэр томьёоны толь

Монгол	Англи	Монгол	Англи
Баталгаа	*proof*	Тэгш сондгойн чанар	*parity*
Беллийн тоо	*Bell number*	Транзитив багасгалт	*transitive reduction*
Босго	*threshold*	Төлөв	*state*
Бүрэн хайлт	*brute force*	Үе	*period*
Гарах хөрш / орох хөрш	*out-neighbour / in-neighbour*	Үндэс	*root*
Дэд мод	*subtree*	Фенвикийн мод	*Fenwick tree (BIT)*
Динамик программчлал	*dynamic programming*	Хоёрын зэргээр үсрэх	*binary lifting*
Зам	*path*	Хэш	*hash*
Ирмэг	*edge*	Хуваалт	*partition*
Лемм	*lemma*	Цикл-гүй чиглэлт граф	*directed acyclic graph (DAG)*
Мод	*tree*	Шугаман суурь	*linear basis*
Нэгтгэх-хайх бүтэц	*disjoint set union (DSU)*	Ээлжилсэн хэсэг	*alternating segment*
Орой	*vertex*	Эрэмбийн статистик	*order statistic*
Оффлайн боловсруулалт	*offline processing*	Стресс тест	*stress test*

Хавсралт B. Дасгалын хариу

Бүлэг 1. A. Numbers on the Board

1.1. $|-3 + 7| - 1 = 3$. Жишээ нь $-3, \ldots, 3$ ба $x = 4$-ийг үлдээж 5, 6, 7-г арилгана. Үлдсэн тоонуудын нийлбэр 4.

1.2. Бүгдийг үлдээхэд нийлбэр 0 болох тул 0 нь сайхан тоо. Нэг ч тоо арилгахгүй.

Бүлэг 2. B. Rhythmic String

2.1. $p = 2$ үед 2, 4, 6, 18; $p = 3$ үед 3, 6, 9, 18 гэж тус бүр 4 элемент үлдэнэ. Хариу $6 - 4 = 2$.

2.2. 1-үүд 5-ын үржвэрүүд дээр байх тул мөр 00001 давтагдана. Үе нь 5, хариу 0.

Бүлэг 3. C. Big XOR

3.1. $L = 3$, $W = 4$, base = 1. $T = 15 \oplus 1 = 14$-ийн 1-ийн тоо 3 буюу сондгой тул $T = 15$ болно. $j = 0$ дээр 1-ийг давхарлаж $T = 12$, $j = 1$ дээр юу ч хийхгүй, $j = 2$ дээр 4-ийг давхарлаж $T = 0$. $S = \{1, 4\}$, $b = [2, 2, 3, 8, 5]$, $m = 14 = 2k$.

3.2. Хоёр утга нэг битээр ялгаатай тул $M \oplus$ base-ийн 1-ийн тооны тэгш сондгой нь эсрэг. Гэтэл d_i-үүдийн XOR-ын 1-ийн тоо үргэлж тэгш (Лемм 3.3).

Бүлэг 4. D. Black, Gray and White

4.1. $k = 9$. n тэгш тул зөвхөн $k = n + 1$ үед бүх орой хар болно (Теорем 4.2), хосолоогүй орой үлдэхгүй.

4.2. $k = 5$: хосолоогүй 5-р орой бүх замын дунд байрлах тул хар 4, саарал 1. $k = 6$: хосолоогүй 3-р орой навч, хар 4, саарал 0. $k = 7$: хосолоогүй 1-р орой навч, хар 4, саарал 0. Хариу $k = 5$.

Бүлэг 5. E. Partition of a Binary String

5.1. Хэсгүүд 01 ба 10, хоёулаа $\ell = 2$. $X = 4$ тул өртөг 8. Ганц хэсэг алга, $c(2) \cdot c(2) = 1$ тул ганц хуваалт бий: {1, 2}, {3, 4}.

5.2. Бүх хэсэг ганц, $X = 0$, өртөг 5. Хуваалтын тоо $\mathrm{Bell}(5) \cdot \mathrm{Bell}(0) = 52$.

Бүлэг 6. F. Minimize the Graph

6.1. nxt(1) = 2, nxt(2) = 3, nxt(3) = 4. $i = 2, 3, 4$ тус бүрд зөвхөн $j = i - 1$ үлдэнэ. Хариу 3: граф нь 1→2→3→4 зам.

6.2. Болохгүй. nxt(1) = 2, nxt(2) = nxt(3) = 4. $i = 4, 5$ үед $r_i = 3$ тул $j = 2, 3$ хоёулаа үлдэнэ. Хариу нь бүх 6 ирмэг.

Бүлэг 7. G. Order Statistics on the Graph

7.1. $c_v = 5 - 2 + 1 = 4$. Хоёр дахь хамгийн бага утга x-ээс багагүй байхын тулд таван утгын дор хаяж дөрөв нь x-ээс багагүй байх ёстой, өөрөөр хэлбэл хамгийн бага нэг утгаас бусад нь.

7.2. $F^3(s)$-ийн утгууд нь $x = 1$: 1111, $x = 3$: 1111, $x = 5$: 0001, $x = 8$: 0000. Тэгэхээр cnt = (2, 2, 2, 3), хариу [3, 3, 3, 5].